Specifying the Generative Relational Research Protocol

Architecture, Conformance, and Deployment over Existing Repositories

Wanhong Huang
huangwanhong@serendip.ngo

Abstract

This paper specifies a protocol for recording the evolution of a shared understanding, and states the conditions an implementation must satisfy to conform. The companion paper established that the artefact has weakened as evidence of a producer's capacity, that a record of the process inherits the same fabrication problem unless its credibility follows from distribution across parties, and that earlier systems representing scholarly reasoning failed on the cost of capture in place of the adequacy of their notations. Sixteen requirements followed. The present paper discharges them. It fixes a three-part model in which events are captured automatically, transitions are registered by a responsible party other than the proposer, and artefacts are referenced; it factors the type of a transition into independent dimensions with small closed vocabularies, binding relations, contributor roles and provenance to deployed standards; it specifies attestation, attribution of absorbed content, parent links and divergence without any merge operation; it fixes disclosure classes bound to declared grounds, monotone release, and the separation of personal content from the attested skeleton so that redaction leaves the log intact; it specifies identity as a key with optional institutional binding, sealed registration separating a claim from its disclosure, propagation between independent implementations, durability, and an amendment procedure. Three conformance tiers are defined, the lowest of which is worth adopting by a single participant, and a deployment over version control and existing repositories is given as one implementation among several. The specification is published as version 0.1 and is expected to be revised through use.

Keywords. protocol specification; trajectory records; attestation; federation; conformance

Draft. This is a working draft. It is circulated for comment, and its arguments, formulations, and open questions are subject to revision. Correction and criticism are welcome at huangwanhong@serendip.ngo.

Use of generative artificial intelligence. Generative AI assistants (Anthropic's Claude and OpenAI's ChatGPT) were used extensively in preparing this manuscript: for literature survey, for drafting prose from the author's specifications, for adversarial review of the argument, for the construction of the figures, for consistency and citation auditing, and for typesetting. Formulations and objections arose in the course of that exchange as well as from the author. Every claim, argument, and citation was reviewed and decided by the author, who is responsible for the content and for any errors. Bibliographic details are to be verified against the primary sources before reliance.

Suggested citation: Wanhong Huang, *Specifying the Generative Relational Research Protocol: Architecture, Conformance, and Deployment over Existing Repositories*, working draft, 2026.

1 Introduction

This section states what the paper specifies, what it inherits, what it deliberately excludes, and how the specification is versioned. Its role is to fix the paper's undertaking before any definition is given. Its structure treats the object specified, the relation to the companion paper, the exclusions, the versioning commitment, and the structure of the argument. Its method is expository, and every claim it makes is established elsewhere.

This paper specifies an interaction protocol for records of research in progress. The protocol fixes which acts are admissible, what each act does to a shared state, which party may perform each act, and what an implementation must satisfy to conform. It fixes no evaluation, no ranking and no interpretation, and it prescribes no research practice.

The unit of record is the transition between two identified states of an understanding. A transition names the state it altered, carries a typed act performed by a party, is registered by a different party, and links to any content it absorbed from elsewhere. A trajectory is the resulting directed graph, and what is ordinarily called the current state of a piece of work is a view derived from that graph in place of a stored object.

The companion paper establishes the problem and derives the requirements. Three of its results govern everything specified here and are stated without re-argument.

An artefact carried evidence of its producer's capacity because producing one was costly in a discriminating way (Spence, 1973), and a uniform fall in that cost weakens the evidence while leaving the artefact and the systems reading it in place.

A record of a process inherits the same weakness where one party produces it alone, so the credibility of a trajectory record follows from its distribution across parties who did not coordinate, and from no property of its content.

Systems adequate to represent scholarly reasoning have existed since 1970 (Kunz and Rittel, 1970; Conklin and Begeman, 1988; Buckingham Shum et al., 2000) and were not adopted, because the work of recording fell on the party who gained least from the record (Grudin, 1988). A design succeeding where they failed differs in the economics of recording and not in the expressiveness of its notation.

Sixteen requirements follow from these results and from the material treated in the companion paper's later sections. They are reproduced in Section 2 and each is discharged at a stated point below.

Four exclusions are fixed here because they explain absences a reader will otherwise take for omissions.

The specification defines no quantity over participants or over trajectories, and admits no total order over either. An implementation may compute such a quantity, and does so outside conformance and on its own account.

The specification requires no analytical capability. Every conformant operation is performable with a text editor and a version-control system, and model-assisted capture is an implementation convenience whose absence changes nothing about conformance.

The specification defines no merge operation. Two revisions of a concept do not compose, no conflict region localises, and no test decides the result, so integration appears only as a synthesis referencing several parents or as an absorption carrying attribution.

The specification states no rules of conduct, no membership criteria, no retention periods and no consent requirements. These belong to the operating charter of a community, which the protocol references and does not interpret.

The protocol specified here is published as the Generative Relational Research Protocol, version 0.1. The version is stated in the paper for two reasons.

A specification without a version and an amendment procedure either ossifies or is revised by whoever holds informal power. Section
refsec:amendment fixes the procedure, and the section on conformance fixes what an implementation declares when it claims conformance to a version.

And the choice is reflexive. The companion paper holds that a design of this kind cannot be settled in advance and is corrected through use. A specification published as finished would contradict the account it specifies, and the version number is the visible form of that commitment.

Section 2 fixes the terms and reproduces the inherited requirements. Section 3 divides responsibility between the protocol and the systems it is layered over. The model is given in the sections on events, transitions and artefacts, on identifiers, on the act vocabulary and its bindings, on attestation, on attribution and absorption, and on parent links and divergence. Disclosure and identity follow, then propagation, durability, the charter interface and amendment. The closing sections fix conformance tiers, give one deployment in detail, work three examples, and state the failure modes and the boundaries of the specification.

2 Scope, Terms, and the Requirements Inherited from the Companion Paper

This section fixes the terms used throughout and reproduces the requirements the specification discharges. Its role is to make the paper auditable: a reader should be able to take each requirement, find the section that satisfies it, and judge whether it does. Its structure treats the conformance vocabulary, the terms, the four levels within which the protocol sits, and the requirements table. Its method is stipulative, and the justification of every requirement lies in the companion paper.

2.1 The Conformance Vocabulary

The words *must*, *must not*, *should* and *may* carry the force fixed for them in specification practice (Bradner, 1997). A conformant implementation satisfies every *must* at the tier it declares. A *should* may be departed from where the implementer states a reason. A *may* carries no obligation.

Requirements are stated in the blocks of this paper and are numbered by section. Where a requirement applies only at a stated conformance tier, the tier is named in the requirement.

2.2 Terms

Definition 2.1. **(Trajectory, state, transition).** *A* state *is an identified condition of a shared understanding, addressable by an identifier and referenceable by other records. A* transition *is a change from one identified state to another, produced by a typed act performed by a party and registered by a party. A* trajectory *is the directed graph whose vertices are states and whose arcs are transitions.*

Definition 2.2. **(Event, artefact).** *An* event *is an automatically generated record that something occurred: a meeting was held, a comment was posted, a file changed, a permission was altered. An* artefact *is a produced object referenced by a record: a document, a dataset, a recording, a transcript, a release.*

Definition 2.3. **(Attestation, registration).** Registration *is the act by which a transition enters the log.* Attestation *is registration performed by a party other than the one who proposed the transition, recorded with that party's identifier and signature.*

Definition 2.4. **(Disclosure class, ground).** *A* disclosure class *is a value governing to whom a record may be disclosed, enforced by an implementation and interpreted by an operating charter. A* ground *is the declared reason for restricting disclosure, which determines both the object restricted and what remains disclosable.*

Definition 2.5. **(Operating charter).** *An* operating charter *is a document held by a community, stating which disclosure classes exist, who belongs to them, what conduct is required, what retention applies, and what identity assurance is demanded. The protocol references a charter by identifier and interprets none of its content.*

2.3 The Levels within Which the Protocol Sits

The companion paper separates external law, operating norms, the protocol, and applications. The separation is carried here in one operative form.

Requirement 2.6. **(Capacity and content).** *The protocol fixes the capacity and never the content. A field whose values an implementation enforces without interpreting them belongs to the protocol; the meaning assigned to those values belongs to the operating charter. No substantive rule of conduct, membership, retention or consent appears in the specification.*

2.4 The Inherited Requirements

Table 2.1 reproduces the sixteen requirements derived in the companion paper, with the sections of this paper that discharge each. The table is the paper's audit surface, and a reader who finds a requirement undischarged has found a defect.

Three of the requirements constrain the specification in ways worth stating in advance, since they exclude designs a reader may expect.

Table 2.1: The inherited requirements and the sections discharging them

	Requirement	Discharged in
R1	Recording is a byproduct of acts performed for their own sake	transitions, conformance
R2	Deployed vocabularies are bound and never redefined	bindings
R3	A transition is registered by a party other than its proposer	attestation
R4	The unit of record is a transition referencing an identified prior state	transitions
R5	The log is append-only; current state is derived; edits are detectable	transitions, integrity
R6	Divergence is recorded and no merge operation is defined	divergence
R7	The complete record is portable and continuable elsewhere	propagation, durability
R8	An entry path exists that is cheap to attempt and externally verifiable	act vocabulary
R9	Absorbed content is attributed to its originator, without veto	attribution
R10	No scalar quantity is defined over participants or trajectories	scope, conformance
R11	Disclosure is monotone, with scheduled release and no withdrawal	release
R12	Personal content is separable, so redaction leaves the log intact	integrity
R13	Records are durable: plain formats, distributed custody, deposit	durability
R14	The protocol is implementable with no analytical capability present	substrates, conformance
R15	The core tier is worth adopting by a single participant	conformance
R16	A release can emit the artefact the incumbent reward system accepts	release

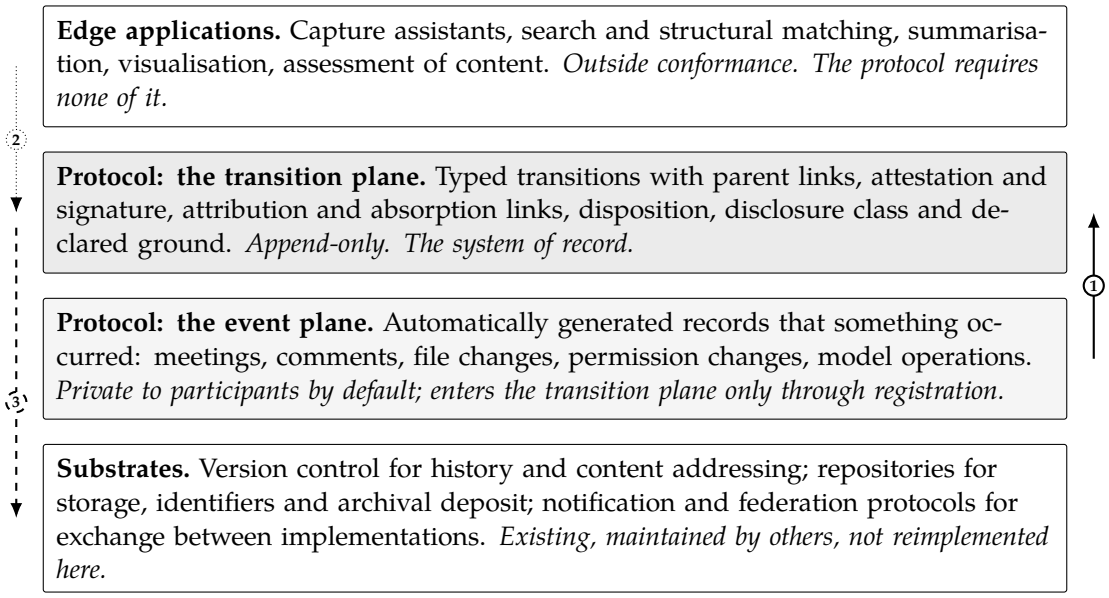
R1 excludes any act that exists solely so that the record should exist. Every act in the vocabulary of Section
refsec:vocabulary is one a participant performs for a reason of their own, and an act failing this test is placed in an optional tier or removed.

R10 excludes progress indicators, contribution scores, activity counts and reputation values from the specification. It is the requirement most likely to be violated by an implementation acting in good faith, and the section on conformance states the test.

R14 excludes any dependence on model-assisted capture, matching or summarisation. Such capability is admitted as an application over the record and is required by nothing in the protocol.

2.5 Summary of Results

This section has fixed the conformance vocabulary, the terms trajectory, state, transition, event, artefact, registration, attestation, disclosure class, ground and operating charter, and the operative form of the level separation in Requirement **??**. Table 2.1 reproduces the sixteen inherited requirements with the section discharging each, and three of them are singled out as constraining the specification against designs a reader may expect: the byproduct condition, the exclusion of scalar quantities, and independence from analytical capability.



(1) *registration by a responsible party promotes an event to a transition.* (2) *applications read and write records without being required by them.* (3) *the protocol delegates storage, identifiers, history and transport.*

Figure 3.1: The division of responsibilities. The transition plane is the system of record and is append-only; the event plane is captured automatically and is disclosed only through registration; substrates supply what already exists. The protocol adds the transition plane and the rules governing promotion into it, and adds nothing else.

3 The Division of Responsibilities between the Protocol and Its Substrates

This section fixes what the protocol supplies and what it delegates. Its role is to bound the specification: a protocol that reimplements storage, identifiers, history or transport acquires obligations it cannot discharge and competes with systems that are maintained by others. Its structure treats the three planes of the architecture, what each substrate supplies, the requirement of substrate independence, and the two properties that must be obtained from a substrate rather than assumed. Its method is derivation from the inherited requirements, and the architecture is illustrated in Figure 3.1.

3.1 The Three Planes

The architecture has three planes and one delegated layer.

The *event plane* holds automatically generated records that something occurred. It is cheap to populate, may be exhaustive, and carries no epistemic claim. It is also the plane on which a surveillance risk falls, since a complete log of who attended, who commented and whose files changed is a monitoring record by construction.

> **Requirement 3.1. (Privacy of the event plane).** *Records on the event plane are disclosed to participants of the trajectory by default and to no other party. An event is disclosed beyond that set only through a transition that references it, and an implementation must not export, index or publish the event plane as such.*

The *transition plane* is the system of record. It is append-only, it holds typed transi-

tions with their attestations and links, and everything the protocol asserts about credibility, attribution and portability concerns this plane alone.

The *artefact references* are pointers to produced objects held elsewhere, with the objects themselves stored by substrates.

Above these, applications read and write records. They are outside conformance, for the reason fixed as R14.

3.2 The Provision of Each Substrate

Three substrates are assumed and none is specified here.

Version control supplies an append-only history with content addressing in the manner of a hash tree (Merkle, 1988), so that a record's integrity is checkable and a retroactive alteration is detectable, and supplies the transport by which a complete record is copied and continued elsewhere.

Repositories supply storage for artefacts under packaging conventions such as those developed for research objects (Soiland-Reyes et al., 2022), persistent identifiers that resolve independently of any one operator, and archival deposit with succession arrangements.

Notification and federation protocols supply the exchange of messages about resources between implementations that share no operator (Confederation of Open Access Repositories, 2022; Webber et al., 2018).

Claim 3.2. *The protocol contributes the transition plane and the rules governing promotion of events into it. Every other capability the architecture requires already exists, is maintained by parties with the resources to continue maintaining it, and is delegated. A design reimplementing those capabilities would inherit their maintenance and would compete with their installed base, which the adoption argument of the companion paper forbids.*

3.3 Substrate Independence

Delegation must not become dependence on one product.

Requirement 3.3. (Substrate independence). *The specification names capabilities and never products. A conformant implementation may satisfy a capability by any means, and must state which system supplies each. No conformance condition is expressed in terms of a particular version-control system, repository service, identifier scheme or federation protocol.*

The requirement has a cost, stated plainly. It prevents the specification from prescribing a file layout or a wire format, and interoperability between implementations therefore rests on the record model and the bindings of Section
refsec:bindings in place of a byte-level agreement. Section
refsec:propagation states what two implementations must agree on to exchange records, which is the minimum this position permits.

3.4 The Two Properties That Must Be Obtained and Not Assumed

Two properties are commonly assumed of substrates and are commonly absent, and both are load-bearing here.

The first is identifier resolution under relocation. An identifier that resolves through the original operator's service ceases to resolve when participants leave, so a record that has been carried elsewhere holds references it cannot follow, and portability is nominal. A conformant implementation must record, for every referenced state and artefact, an identifier that continues to resolve when the record is held by another party.

The second is content addressing of the transition plane. Detecting a retroactive alteration requires that each transition's identifier derive from its content and from its parents, so that altering an early transition invalidates everything after it. Where a substrate supplies this, the protocol inherits tamper evidence at no cost; where it does not, the implementation must supply it.

> **Requirement 3.4. (Resolution and content addressing).** *Every reference to a state or an artefact must use an identifier that resolves independently of the implementation that created it. Every transition must carry an identifier derived from its content and from the identifiers of its parents.*

3.5 Summary of Results

This section has fixed the architecture as three planes over delegated substrates. The event plane is captured automatically, is private to participants by Requirement **??**, and reaches disclosure only through registration. The transition plane is the append-only system of record and carries every claim the protocol makes. Version control, repositories and federation protocols supply history, storage, identifiers, deposit and transport, and Claim **??** states that the protocol contributes the transition plane and the rules of promotion into it and nothing else. Requirement **??** forbids naming products in conformance conditions, at the stated cost that interoperability rests on the record model rather than a wire format. Requirement **??** fixes the two properties that must be obtained from a substrate and not assumed: identifiers that resolve after relocation, and content addressing of the transition plane.

4 The Event, Transition and Artefact Model

This section fixes the three kinds of record the protocol defines and the rules by which one becomes another. Its role is to discharge the requirements concerning the unit of record, the append-only log and the economics of capture. Its structure treats the three kinds, the promotion of an event into a transition, the append-only condition and the derivation of state, the granularity condition, and the separate class of administrative operations. Its method is definition followed by requirement, with the reason for each requirement stated in one sentence and its full justification left to the companion paper.

4.1 The Three Kinds of Record

Definition 4.1. (The three record kinds). *An* event *records that something occurred and asserts nothing about an understanding. A* transition *records that an identified state of an understanding became another, through a typed act performed by a party and registered by a party. An* artefact reference *records that a produced object exists, with an identifier resolving to it.*

The division carries the economics of capture. Events are generated by the systems participants already use, so their cost is near zero and their volume may be large. Transitions require a human judgement and a registering party, so their cost is not zero and their volume is small. Artefact references are produced as a side effect of depositing anything.

Requirement 4.2. (Recording as byproduct). *Every act the protocol defines must be one a participant performs for a reason of their own. An implementation must not require any act whose only purpose is that a record should exist, and must not make conformance depend on such an act.*

The requirement is the one that determined the whole design, since the systems surveyed in the companion paper failed on it and not on the adequacy of their notations (Grudin, 1988).

4.2 The Promotion of an Event into a Transition

Events do not enter the transition plane by themselves, and nothing on the event plane carries an epistemic claim.

Requirement 4.3. (Promotion). *A transition enters the log only by the act of a party who accepts responsibility for it. An implementation may propose candidate transitions by any means, including analysis of the event plane by a model, and a proposal becomes a transition only when a responsible party registers it. The record must show which party registered, and must record that a proposal originated from an automated analysis where it did.*

Three consequences follow and are stated so that an implementer does not have to derive them.

Cheap confirmation is permitted and reflexive confirmation is not. An implementation may present a candidate for acceptance in one action, and the action must be affirmative: a default that registers unattended proposals violates the requirement, because the record’s credibility rests on registration meaning something.

Volume on the event plane is harmless. A meeting of two hours that alters no state produces events and no transitions, and nobody decides what deserved to be kept, since the recording unit decides it.

A model may occasion a transition and may not author or register one. The record marks such an origin, which allows a later reader to weigh the transition accordingly and keeps the arrangement compatible with authorship rules that exclude non-human authors (International Committee of Medical Journal Editors, 2023).

4.3 The Append-Only Condition and the Derivation of State

Requirement 4.4. (Append-only log and derived state). *The transition plane is append-only. A recorded transition must not be altered or removed; a correction is a further transition referencing the one corrected. The current state of a trajectory must be derived from the log on demand, and must not be stored as an authoritative object alongside it.*

The second sentence of the requirement is the one most often violated in practice, since a stored current state is convenient and fast. Where a stored snapshot is authoritative, the log becomes a secondary artefact that may drift from it or be edited to match it, and every claim the protocol makes about credibility fails at once. An implementation may cache a derived view freely, provided the cache is reconstructible from the log and is marked as derived.

4.4 The Granularity Condition

Requirement 4.5. (Granularity). *A transition must reference a specific identified prior state. A record attached to a project, a repository, a document or a person as a whole is not a transition, and an implementation must not admit one.*

The requirement explains a documented failure. Commentary systems attach a remark to an artefact, so no state is superseded and the relation between the remark and any later change exists only in the memory of the author. A protocol admitting project-level records would reproduce that outcome while claiming to have avoided it.

4.5 Administrative Operations

Some records concern the arrangement rather than the understanding: a disclosure class was changed, a permission was altered, an attribution was recorded, a fork was declared.

Requirement 4.6. (Separation of administrative operations). *Operations on the record itself must be recorded, must carry the identifier of the party performing them, and must be distinguished from transitions by a mandatory kind field. An implementation must not present administrative operations and transitions as records of the same kind.*

Two reasons. Administrative activity would otherwise inflate the apparent generativity of a trajectory, which matters because the protocol computes no quantity and readers will nonetheless count. And the admissibility condition on positions, in the companion paper, requires that the acts constituting a position be recorded and attributable; a disclosure holder who could alter a class without leaving a record would occupy a position that no participant could inspect.

4.6 Summary of Results

This section has fixed the three record kinds in Definition 4.1, and with them the economics of capture: events are near-free and may be exhaustive, transitions cost a judgement and a registrar and are therefore few, and artefact references fall out of deposit.

Requirement 4.2 forbids any act existing solely so that a record should exist. Requirement 4.3 fixes promotion by a responsible party, permits cheap affirmative confirmation, forbids unattended defaults, and requires model-originated proposals to be marked. Requirement 4.4 fixes the append-only log and forbids an authoritative stored current state, permitting reconstructible caches. Requirement 4.5 fixes granularity and explains why commentary systems have not become epistemic infrastructure. Requirement 4.6 separates administrative operations from transitions by a mandatory kind field.

5 Trajectories, States, and Persistent Identifiers

This section fixes what a state is, how it is identified, and what an implementation must guarantee about identifiers. Its role is to make states addressable, since every other mechanism in the protocol references them: transitions name their prior state, absorption links name a state in another trajectory, disclosure classes attach to records about states, and portability requires that references survive relocation. Its structure treats the state, its identifier, the derivation of views, the boundary of the trajectory, and the treatment of external references. Its method is definition followed by requirement.

5.1 The State

Definition 5.1. (State). *A state is an identified condition of a shared understanding at a point in a trajectory, consisting of the content its participants have recorded together with the identifier under which that content is addressed. A state is created by a transition and is never modified; a change to what participants hold is a further state produced by a further transition.*

The definition leaves the content of a state unspecified, and deliberately. A state may be a paragraph, a formal statement, a diagram, a claim with its qualifications, or a pointer to a section of a document held elsewhere. The protocol requires that it be addressable and does not require that it be of any kind, since a specification prescribing the form of a scholarly claim would be prescribing research practice.

5.2 The Identifier

Requirement 5.2. (State identifiers). *Every state must carry an identifier that is unique within the trajectory, that resolves to the state's content for a party permitted to read it, and that continues to resolve when the record is held by an implementation other than the one that created it. An implementation must not use an identifier whose resolution depends on a service it alone operates.*

The last sentence is the operative one and is the property most often absent in deployed systems. A record carried elsewhere whose references no longer resolve is an archive rather than a continuable trajectory, and portability is nominal.

Two constructions satisfy the requirement and are named without preference. A content-derived identifier resolves by lookup in whatever store holds the record, and remains valid as long as the content does. A persistent identifier issued by a reposi-

tory service resolves through infrastructure maintained by parties independent of the implementation, which is the arrangement the findability principle of open data practice assumes (Wilkinson et al., 2016). An implementation may use both, and where it uses both it must record the correspondence.

5.3 Derived Views

Readers want the current state of a piece of work, a list of open questions, or the sequence of changes to a particular claim. None of these is stored.

> **Requirement 5.3. (Derivation of views).** *Any view of a trajectory, including its current state, must be computable from the log by an implementation that has the log and no other information. An implementation may cache views, and must be able to reconstruct any cached view from the log alone.*

The requirement has a consequence worth stating for implementers. It forbids a view whose computation depends on information held only by the implementation, such as an ordering produced by a proprietary analysis, from being presented as part of the record. Such a view is an application output, and Requirement **??** places it outside the protocol.

5.4 The Boundary of a Trajectory

A trajectory is a set of states and transitions grouped under an identifier, and the grouping is a decision of its participants rather than a fact the protocol determines.

> **Requirement 5.4. (Trajectory boundary).** *A trajectory must carry an identifier, a statement of the question or undertaking it concerns, and the identifiers of its parent trajectories where it was created by divergence from another. Membership of a state in a trajectory must be recorded, and a state may belong to more than one trajectory.*

The last clause admits the ordinary case in which two lines of work share a state. The protocol records the sharing and asserts nothing about which trajectory the state principally belongs to, for the same reason it designates no principal branch.

5.5 External References

States refer to work outside the trajectory: published articles, datasets, code, recordings, standards.

> **Requirement 5.5. (External references).** *A reference to an external work must use a persistent identifier where one exists, and must record the identifier scheme. Where no persistent identifier exists, the reference must record enough bibliographic or descriptive information for the work to be identified by a reader, together with the date on which the reference was made.*

The date requirement addresses a failure that accumulates silently. A reference to a resource that has changed or disappeared is uninterpretable without knowing when it was made, and a trajectory whose value is expected to appear years later will contain many such references.

5.6 Summary of Results

This section has fixed the state as an identified and immutable condition of a shared understanding, with its content left unspecified so that the protocol prescribes no form of scholarly claim. Requirement 5.2 requires identifiers that resolve after relocation and forbids dependence on a single operator's resolution service, with content-derived and repository-issued identifiers named as the two satisfying constructions. Requirement 5.3 requires every view, including the current state, to be computable from the log alone, which places proprietary orderings outside the record. Requirement 5.4 fixes the trajectory boundary, records parent trajectories, and admits a state belonging to several trajectories. Requirement 5.5 fixes external references, including the date on which a reference was made.

6 The Act Vocabulary and the Factorisation of the Record

This section fixes the vocabulary in which a transition is typed. Its role is to supply the part of the specification that has no counterpart in any deployed standard, since provenance and citation vocabularies describe what a process produced and none of them types a change in an understanding. Its structure treats the factorisation, the eight acts, the remaining dimensions, the extension mechanism, and the entry path the vocabulary makes available. Its method is definition, with the argument against a flat taxonomy stated once and its consequences applied throughout. Figure 6.1 gives the complete record.

6.1 The Factorisation

A vocabulary for changes in an understanding invites a long enumeration: separate types for a modified hypothesis, a changed assumption, a reformulated concept, an extended theory, a replaced theory, a split question, a generalised question, a challenged citation, a discovered constraint, and so through several dozen.

Such a list is a flattened product of independent dimensions, and flattening has a cost that is not aesthetic. Agreement between parties classifying the same occurrence declines as the number of categories rises, and a vocabulary in which classification requires deliberation reintroduces the capture cost that Requirement 4.2 excludes.

> **Requirement 6.1. (Factorisation of the type).** *A transition is typed along five independent dimensions, each with a small closed vocabulary: the act performed, the target on which it operated, the relation the posterior state bears to the prior, the trigger occasioning it, and the disposition. An implementation must not define a single enumerated type combining these dimensions.*

The test an implementer should apply when tempted to add a value: a distinction that two competent participants would classify differently more than occasionally belongs in a community's charter and not in the protocol.

6.2 The Eight Acts

Definition 6.2. (**The act vocabulary**). *The acts are eight.* Question *opens a line of inquiry or introduces a new question within one.* Claim *states a position as the content of a state.* Challenge *raises an objection against an identified state.* Transformation *alters a state in response to something, producing its successor.* Decision *records that a direction was taken or abandoned, with the reason.* Connection *relates a state to another state or to an external work.* Verification *reports the outcome of a check performed on a state.* Release *declares a state published at a disclosure class, with the objections standing against it at that moment.*

Three of the eight carry more weight than their brevity suggests.

Decision is what makes an abandoned line interpretable later. A path recorded as abandoned without a reason cannot be revisited by anyone, and the reuse of abandoned work is one of the two concrete benefits the companion paper claims.

Release is a declaration and never a judgement. It asserts that a state is published, and it records which objections stand unresolved at the moment of publication. It does not assert that the objections were answered, and an implementation must not present a release as a certification of quality.

Challenge produces a transition when it is registered, and the state it challenges is not thereby altered. Where the challenge is accepted, a subsequent transformation produces the successor state, and the two transitions are linked by parent references.

Contribution is not among the acts. It types a party's part in an act and is bound to a deployed contributor vocabulary in the following section.

6.3 The Remaining Dimensions

Target names what the act operated on: a question, an assumption, a hypothesis, a concept, a theory, a method, a path, or an artefact. The vocabulary is extensible by a community.

Relation names how the posterior state stands to the prior: extension, modification, replacement, refinement, generalisation, specialisation, transfer. It is bound to a deployed citation-typing vocabulary and is not defined here.

Trigger names the occasion: reflection, literature, experiment, simulation, observation, discussion, an objection, a prior failure, an automated suggestion, or the entry of a new party. Where the trigger refers to something recorded, the reference is included.

Requirement 6.3. (**The disposition vocabulary**). *A transition carries a disposition with exactly three values: accepted, contested, and unresolved. The disposition vocabulary is fixed by the protocol and must not be extended or reduced by an implementation.*

The unresolved value is required and is the reason the vocabulary is fixed. Most objections in theoretical work are never resolved: they stand, and the work proceeds beside them. A record admitting only acceptance and rejection would be systematically false about the fields this protocol most concerns, and would exert pressure toward fabricated closure.

6.4 Extension

Requirement 6.4. (Extension per dimension). *Communities may extend the target and trigger vocabularies by declaring additional values in an operating charter, with the charter identified in the record. The act vocabulary and the disposition vocabulary must not be extended. An implementation encountering an unrecognised target or trigger value must retain the record and must not reject it.*

The asymmetry is deliberate. Targets and triggers are domain-specific, so a mathematics community will want obstruction types and a laboratory will want deviation types. Acts and dispositions are what interoperability rests on, and a record whose act vocabulary varied by community could not be exchanged.

6.5 The Entry Path

The vocabulary supplies the mechanism by which a party holding no position may participate, which the companion paper requires.

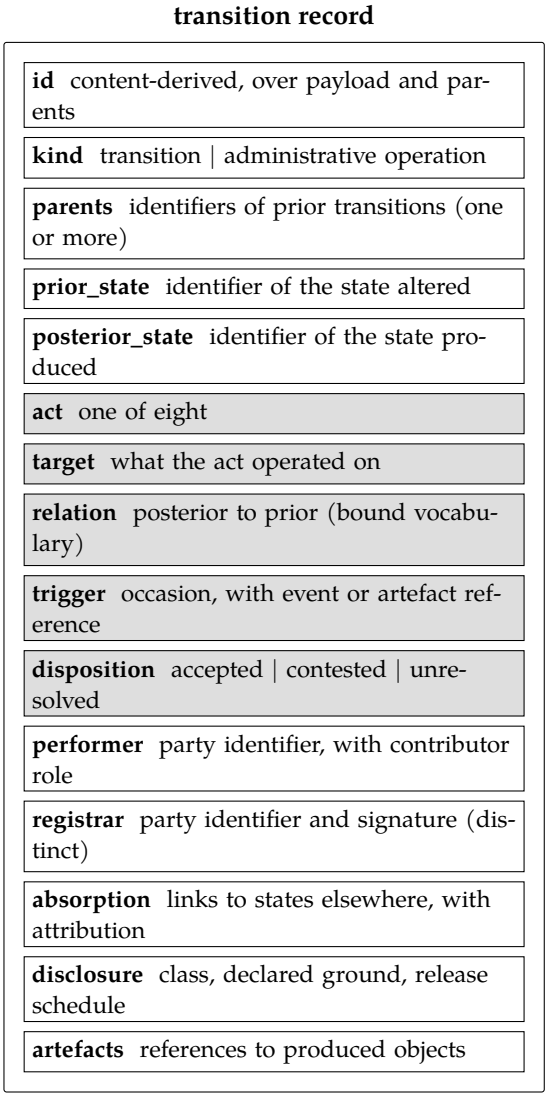
States whose disposition is unresolved constitute a register of open problems: obstacles encountered, objections standing, questions raised without answers. Each is an identified state under Requirement 4.5, and each may be referenced by a party who has performed no prior act in the trajectory.

Requirement 6.5. (Availability of the open register). *An implementation must make available, at each disclosure class, the set of states within that class whose disposition is unresolved, and must permit a party holding no prior position in the trajectory to reference such a state in a challenge, a connection or a verification.*

Whether such a contribution is accepted is decided by the parties holding the state, on the content of the act. The protocol supplies the addressable problem and the admissible act, and supplies no obligation to accept anything.

6.6 Summary of Results

This section has fixed the typing of a transition. Requirement 6.1 factors the type into five independent dimensions and forbids a single enumerated type, with the classification-agreement test stated for implementers. Definition 6.2 fixes the eight acts, and decision, release and challenge are singled out: decision makes abandonment interpretable, release is a declaration recording standing objections and never a certification, and challenge does not itself alter the state it challenges. Requirement 6.3 fixes three dispositions including unresolved, and forbids extension. Requirement 6.4 permits extension of targets and triggers by charter while forbidding it for acts and dispositions, and requires unrecognised values to be retained. Requirement 6.5 makes the register of unresolved states available at each disclosure class and open to parties holding no prior position, which is the entry path the companion paper requires.



Shaded rows are the factored type. *Five independent dimensions with small closed vocabularies replace a single enumerated list of several dozen tags, so that a contributor makes a small number of narrow choices and agreement between parties classifying the same event stays high.*

Extension is per dimension. A community declares additional target or trigger values in its operating charter; the act vocabulary and the disposition vocabulary are fixed by the protocol.

registrar *differs from* ***performer*** *by Requirement 4.3.* ***absorption*** *carries attribution without any power to exclude.* ***disclosure*** *names a ground, and the ground determines what remains disclosable.*

Figure 6.1: The transition record. Fields above the shaded block establish position in the graph; the shaded block is the factored type; fields below carry parties, attribution, disclosure and artefacts. An administrative operation uses the same envelope with the act, target, relation and disposition fields absent and an operation field in their place.

7 Bindings to Deployed Vocabularies

This section fixes what the protocol takes from existing standards and how it takes it. Its role is to discharge the requirement that deployed vocabularies be bound and never redefined, and to keep the specification's new terminology confined to the part that is genuinely new. Its structure treats the principle of binding, the four bindings, the rules governing how a binding is recorded, and the failure cases a binding introduces. Its method is to name a capability, name the class of standard that supplies it, and fix the manner of reference, without naming any standard as the only admissible one.

7.1 The Principle of Binding

Requirement 7.1. (Binding in place of redefinition). *Where a deployed standard defines a vocabulary the protocol requires, the protocol binds to it by identifier and does not restate its terms. New terminology is defined only for the act vocabulary, the disposition vocabulary, and the record structure of Section 6.*

Three reasons, of which the third is decisive. A specification restating an existing vocabulary acquires the obligation to track its revisions. Conformance becomes cheaper for parties already using the standard. And a specification that reinvents a deployed vocabulary is dismissed by the community that maintains it, which is the ordinary fate of standards written in isolation.

7.2 The Four Bindings

Relations between states bind to a citation-typing vocabulary. The relations the protocol requires, extension, modification, replacement, refinement, generalisation, specialisation and transfer, are among those such vocabularies already define for relations between works (Shotton, 2010), and the protocol applies them to relations between states.

Contributor roles bind to a contributor taxonomy (Allen et al., 2019). The protocol requires that a party's part in an act be recordable in a controlled vocabulary, and the deployed taxonomies supply one that publishers already accept, which also serves the compatibility requirement of the closing sections.

Agents, activities and derivation bind to a provenance model (Moreau and Groth, 2013). A transition is expressible in such a model as an activity with an agent and a derivation, and an implementation exporting to a provenance serialisation makes its records readable by tooling that knows nothing of this protocol.

Anchored commentary binds to an annotation model (Sanderson et al., 2017). Where a challenge or a connection is occasioned by a passage of a document or an interval of a recording, the anchor is expressed in the annotation model's selectors, and the act type of this protocol is carried in the annotation's motivation.

Claim 7.2. *Under these four bindings the protocol's own terminology reduces to the eight acts, the three dispositions, the record structure, and the disclosure and attestation fields. Everything else the specification requires is expressed in vocabularies that exist, are maintained by others, and are already implemented in scholarly infrastructure.*

7.3 The Recording of a Binding

A binding that is not recorded is an assumption, and assumptions do not survive relocation of a record.

> **Requirement 7.3. (Recording of bindings).** *A record must identify, for each bound vocabulary it uses, the vocabulary and the version to which its values refer. A value drawn from a bound vocabulary must be recorded as an identifier in that vocabulary and not as a display label.*

The second sentence prevents a common and quiet failure. A record storing the word *extends* rather than the vocabulary's identifier for that relation is uninterpretable when a second vocabulary uses the same word differently, and translation between implementations then depends on a guess.

7.4 The Failure Cases a Binding Introduces

Binding transfers a dependency, and the specification states what happens when the dependency fails.

A bound vocabulary may be revised. The version recorded under Requirement 7.3 makes the revision detectable, and an implementation must treat records referring to an earlier version as valid and must not rewrite them.

A bound vocabulary may be withdrawn or may cease to be maintained. In that case the identifiers in existing records remain the authoritative statement of what was meant, and a community may declare a replacement vocabulary in its charter with a mapping from the withdrawn identifiers. The protocol does not require the mapping to be complete, since a partial mapping preserving what can be preserved is better than a rewrite that silently changes what records assert.

A bound vocabulary may lack a value a community needs. Where the missing value is a relation or a contributor role, the community declares an extension in its charter and records it as such, so that a reader can distinguish a bound value from a local one.

> **Requirement 7.4. (Local values marked).** *A value not drawn from a bound vocabulary must be marked as local and must identify the charter that defines it. An implementation must not present a local value as though it were drawn from a bound vocabulary.*

7.5 Summary of Results

This section has fixed the manner in which the protocol uses existing standards. Requirement 7.1 binds rather than redefines, confining new terminology to the acts, the dispositions and the record structure. Four bindings are fixed: relations to a citation-typing vocabulary, contributor roles to a contributor taxonomy, agents and derivation to a provenance model, and anchored commentary to an annotation model, with the act type carried in the annotation motivation. Claim 7.2 states the consequence: the protocol's own vocabulary is small, and everything else is expressed in maintained standards. Requirement 7.3 requires the vocabulary and version to be recorded and values

to be stored as identifiers rather than labels. Three failure cases are treated, and Requirement 7.4 requires local values to be marked and attributed to the charter defining them.

8 Registration, Attestation, and Signature

This section fixes how a transition enters the log and what its entry asserts. Its role is to discharge the requirement on which the record's credibility rests, since a trajectory held and produced by one party is as cheaply fabricated as the artefact whose evidential function it would inherit. Its structure treats registration and attestation, the signature, what attestation asserts and what it does not, the self-registered case, and the treatment of disputes and revocation. Its method is definition followed by requirement, with the limits of the mechanism stated as fully as its powers.

8.1 Registration and Attestation

Requirement 8.1. (Attestation). *A transition must record the party who performed the act and the party who registered it. Where those parties are distinct, the transition is attested. An implementation must record the registering party's identifier and a signature over the transition's content and parents, and must not permit registration in which the registering party is unidentified.*

The requirement is satisfied by an act that costs the registrar very little: reading a proposed transition and confirming it. That cheapness is required rather than convenient, since the registrar performs work whose benefit accrues to someone else, which is the disparity that defeated earlier systems (Grudin, 1988).

Requirement 8.2. (Independence of the registrar). *An implementation must not register a transition automatically on behalf of a party, and must not permit a party to hold the credentials of another for the purpose of registration. Where a transition is registered by a party acting under an arrangement with the performer, the arrangement must be recorded.*

The second sentence addresses the ordinary case of a supervisor registering a student's transitions as a matter of routine. The arrangement is legitimate and its effect on the record's independence is real, so it is recorded rather than forbidden.

8.2 The Signature

A signature binds an identifier to a content (Merkle, 1988), and the identifier is a key in the sense fixed in Section
refsec:identity.

Requirement 8.3. (Signature coverage). *A signature must cover the transition's content, its parent identifiers, its prior and posterior state identifiers, and its disclosure class. A signature must not cover fields that a later operation may lawfully alter, and an implementation must state which fields those are.*

The second sentence prevents a design error. Disclosure may widen over time under

scheduled release, and content may be redacted where a participant withdraws personal material. A signature covering fields subject to lawful later change would be invalidated by ordinary operation, and implementations would then either forbid the operation or ignore the invalidation, both of which defeat the purpose.

8.3 The Content of an Attestation

Claim 8.4. *An attestation asserts that an identified party registered a transition at a time. It asserts nothing about whether the change was an improvement, whether the resulting claim is true, or whether the registering party understood it. Assessment of content lies outside the protocol and is performed by readers.*

The claim is worth stating in the specification, because implementations will be tempted to present an attested transition as a verified one and readers will make the inference unaided.

Three limits follow and are recorded here rather than in a closing section, since an implementer must know them.

Credibility appears only where more than one party is present. A trajectory maintained alone carries no attestation, and Requirement 8.6 requires it to be marked accordingly.

Collusion is not eliminated. Parties who coordinate can produce a mutually attested fabrication, and the cost of doing so rises with the number of independent parties and with the visibility of their other work. The mechanism raises the cost of fabrication and does not reduce it to zero.

Counting is forbidden. The natural response to the previous limit is to measure attestation depth, and that measure is a scalar over trajectories which Requirement **??** excludes and which reciprocal registration would farm within a small group.

Requirement 8.5. **(No aggregation over attestations).** *An implementation must not compute, store or present any aggregate over attestations, including counts, depths, ratios and derived scores, as part of a conformant record. Attestation is a property of an individual transition with two values.*

8.4 The Self-Registered Case

A single participant using the protocol alone obtains a usable record and no evidential weight, and the specification must not obscure the difference.

Requirement 8.6. **(Marking of unattested transitions).** *A transition whose performing and registering parties are the same must be recorded as unattested, and an implementation must present it as unattested wherever it presents attested transitions. An implementation must not describe a wholly self-registered trajectory as verified, corroborated or independently recorded.*

The requirement protects the mechanism's meaning at the point where it would otherwise be diluted, since the personal tier is the tier most people will use first.

8.5 Disputes and Revocation

A registrar may come to believe that a transition they registered misdescribes what occurred.

Requirement 8.7. (Withdrawal of an attestation). *An attestation must not be deleted. A registrar who wishes to withdraw an attestation records a further transition of the challenge act, referencing the transition attested and stating the ground of withdrawal. The original attestation remains in the log with the withdrawal linked to it.*

The rule follows from the append-only condition and has a consequence worth naming. A reader encountering a withdrawn attestation sees both the original act and its withdrawal, which is more informative than either a deletion or a silent correction, and which places the dispute in the record where later readers can weigh it.

8.6 Summary of Results

This section has fixed registration and attestation. Requirement 8.1 requires both performing and registering parties to be recorded, with a signature over content, parents, states and disclosure class. Requirement 8.2 forbids automatic registration on a party's behalf and requires standing arrangements to be recorded. Requirement 8.3 fixes signature coverage and excludes fields subject to lawful later change, naming scheduled release and redaction as the two such operations. Claim 8.4 fixes what an attestation asserts, and three limits are recorded: credibility requires at least two parties, collusion is made costly rather than impossible, and counting is forbidden by Requirement 8.5. Requirement 8.6 requires self-registered transitions to be marked unattested and forbids describing a self-registered trajectory as corroborated. Requirement 8.7 forbids deletion of an attestation and fixes withdrawal as a further recorded act.

9 Attribution, Absorption Links, and Credit without Exclusion

This section fixes how a party's part in a transition is recorded, and how content taken from one line of work into another is credited. Its role is to discharge the requirement that absorbed content be attributed to its originator without any power to exclude, which is the mechanism distinguishing a generative arrangement from an open competition. Its structure treats attribution within a transition, the absorption link, the exclusion of veto rights, disputes, and what the record does not settle. Its method is definition followed by requirement, with the legal position taken from the companion paper and not re-argued.

9.1 Attribution within a Transition

Requirement 9.1. (Attribution of an act). *A transition must record the identifier of the party who performed the act. Where several parties contributed to it, each must be recorded with a role drawn from the bound contributor vocabulary* (*Allen et al., 2019*)*. An implementation must not record a contribution without a party, and must not record a party without a role where more than one party is present.*

Attribution attaches to an act and not to a finished work, which is the difference between this record and a contributor statement on a publication. A contributor statement says that a person contributed to a paper. This record says which change in the content of a claim a person produced.

9.2 The Absorption Link

Definition 9.2. (Absorption). *Absorption is the taking of content from a state in one trajectory into a transition in another. An absorption link records the identifier of the state absorbed from, the identifier of the party who produced it, and the transition into which the content was taken.*

Requirement 9.3. (Recording of absorption). *Where a transition takes content from a state outside its own trajectory, the transition must carry an absorption link identifying that state and the party who produced it. An implementation must present absorption links alongside the transition wherever the transition is displayed.*

The mechanism is what makes the difference between two arrangements visible in the record. In an open competition, unselected proposals are discarded and their content, where it is used, is used without trace. In an arrangement conforming to this protocol, content taken from an unselected line appears as an absorption link naming the party who produced it. Whether a given arrangement absorbs or discards is therefore checkable from its record, and the check is performed by readers rather than by the protocol.

9.3 Credit without Exclusion

Requirement 9.4. (No veto). *An absorption link confers attribution and confers no right to prevent, condition or reverse the use of the content. An implementation must not provide a mechanism by which the party named in an absorption link can block a transition, require approval before absorption, or withdraw content already absorbed.*

The requirement is deliberate and its ground is stated in the companion paper. Rights to exclude, multiplied across many small contributions, produce fragmentation in which downstream work requires many permissions and therefore does not occur (Heller and Eisenberg, 1998). The model followed here is the moral-rights side of intellectual property (World Intellectual Property Organization, 1979), where attribution and integrity are secured independently of any power over exploitation.

A consequence worth naming for implementers: a party who does not wish their state to be absorbed has one instrument, which is the disclosure class of that state. Absorption operates on what has been disclosed to the absorbing party, and a party who discloses to a class has accepted that members of the class may build on the content with attribution.

9.4 Disputes over Attribution

Attribution will be disputed, and the protocol treats a dispute as a record rather than as a matter to adjudicate.

> **Requirement 9.5. (Contested attribution).** *A party who holds that an attribution is incorrect, or that an absorption occurred without a link, records a transition of the challenge act referencing the transition concerned and stating the ground. An implementation must not delete or alter the disputed record, must link the challenge to it, and must present both wherever either is presented.*

The protocol supplies no procedure for resolving such a dispute and no party empowered to resolve it. Resolution, where it occurs, occurs in the operating charter of the community or outside the arrangement entirely, and the record's contribution is that both positions are visible with their dates.

9.5 The Matters the Record Does Not Settle

Three limits are fixed as constraints on what an implementation may claim.

An attribution records that a party performed an act. It does not record that the party originated the idea the act expressed, since the same idea may be arrived at independently and the record adjudicates nothing between independent arrivals.

An absorption link records that content was taken. It does not record how much the taking mattered to what followed, and an implementation must not compute or display any measure of the influence of an absorbed state.

And the presence of absorption links in a trajectory does not establish that the arrangement was generative. It establishes that content moved with attribution, which is a necessary condition and not a sufficient one, for the reason given in the companion paper: two trajectories may coincide in their recorded appearance while the relations producing them differ entirely.

9.6 Summary of Results

This section has fixed attribution as attaching to an act rather than to a finished work, with contributor roles drawn from a bound vocabulary. Definition 9.2 fixes absorption, and Requirement 9.3 requires an absorption link naming the state absorbed from and the party who produced it, which makes the difference between an absorbing and a discarding arrangement checkable from the record by readers. Requirement 9.4 excludes any veto, following the moral-rights model and avoiding fragmentation, and leaves the disclosure class as the only instrument by which a party controls absorption. Requirement 9.5 fixes contested attribution as a further recorded act with no adjudicating party. Three limits are fixed: attribution is not origination, absorption carries no measure of influence, and the presence of absorption links does not establish generativity.

10 Parent Links, Divergence, and Synthesis

This section fixes the structure of the trajectory graph. Its role is to discharge the requirement that divergence be recorded and that no merge operation be defined, and to state what an implementation must do where a line of work proceeds in several directions at once. Its structure treats parent links and the graph, divergence, synthesis, the absence of merge, and the treatment of identity across a divergence. Its method is definition followed by requirement, with the argument against merging taken from the companion paper.

10.1 Parent Links and the Shape of the Graph

Requirement 10.1. (Parent links). *A transition must carry the identifiers of one or more parent transitions, except for the transition creating a trajectory, which carries none. The graph formed by transitions and parent links must be acyclic, and an implementation must reject a transition whose parents include a descendant of itself.*

The acyclicity condition is not a formality. A record permitting cycles admits a history in which a state precedes and follows itself, and every derived view, including the current state, becomes ill-defined.

Ordering follows from the graph and not from timestamps. Two transitions with no path between them are unordered, whatever their recorded times, and an implementation must not present unordered transitions as though one preceded the other. Recorded times are evidence about when parties acted and are not the structure of the history.

10.2 Divergence

Definition 10.2. (Divergence). *A divergence occurs where two or more transitions share a parent and produce distinct successor states. Each resulting line is a branch, and the branches are recorded as such.*

Requirement 10.3. (No principal branch). *An implementation must not designate one branch of a divergence as principal, default, canonical or current, and must not order branches by any property of their content, size or activity. Where a reader requires a single view, the implementation must require the reader to select a branch.*

The requirement follows from the exclusion of scalar quantities and from the substantive position of the companion paper: in inquiry a fork is frequently the correct outcome, and a protocol designating a principal line would adjudicate a question about the content that it has no standing to decide.

An implementation may of course display branches in some order on a screen. What it must not do is record or export an ordering as part of the trajectory, or present one branch with the marks of authority.

10.3 Synthesis

Definition 10.4. (Synthesis). *A synthesis is a transition carrying two or more parents drawn from distinct branches, producing a state whose content the performing party has composed from the states those branches reached.*

A synthesis is an ordinary transition of the transformation act with several parents. It is performed by a party, is registered like any other, and asserts nothing beyond what its performer composed. The branches it draws on continue to exist and are not closed by it, since a synthesis is a further state and not a resolution of the divergence.

10.4 The Absence of Merge

Requirement 10.5. (No merge operation). *The protocol defines no operation that combines two states automatically. An implementation must not compute a combined state from two divergent states, must not present such a computation as a transition, and must not use the term merge for synthesis or for absorption.*

The ground is that the preconditions for mechanical combination are absent. Version control merges because changes to distinct regions of a text compose, because a conflict can be localised, and because a test decides whether the result works. Two revisions of a concept satisfy none of the three.

The terminological clause is practical rather than pedantic. The protocol is layered over substrates in which merge names an operation that does exist, and an implementation using the word for synthesis would lead every implementer to expect behaviour the protocol does not have.

10.5 Identity across a Divergence

Requirement 10.6. (Identity is recorded, not adjudicated). *An implementation must not determine whether the states reached by two branches are versions of the same object. Where a reader asks whether a later state is a version of an earlier one, the implementation must answer with the chain of transitions connecting them, and must not answer with a judgement.*

The requirement fixes a boundary the protocol has held throughout. Whether a theory that has abandoned an assumption, reformulated a concept and narrowed its scope remains the same theory is a question for the parties concerned. The protocol supplies the chain that makes the question answerable and declines to answer it.

Two practical consequences. A trajectory may contain several live branches indefinitely, and this is a normal condition rather than an unfinished one. And a reader arriving at such a trajectory must choose among the branches, with the protocol supplying no basis for the choice, which is a cost stated here and not concealed.

10.6 Summary of Results

This section has fixed the graph. Requirement 10.1 requires parent links, forbids cycles, and fixes ordering by the graph in place of timestamps, with unordered transitions not to be presented as ordered. Definition 10.2 fixes divergence and Requirement 10.3 forbids designating any branch principal, default or canonical, or exporting an ordering over branches. Definition 10.4 fixes synthesis as a transformation with several parents which does not close the branches it draws on. Requirement 10.5 excludes any merge operation, on the ground that composition, conflict localisation and a deciding test are all absent, and forbids the term merge for synthesis or absorption. Requirement 10.6 fixes that identity across a divergence is recorded as a chain and never adjudicated, at the stated cost that a reader must choose among live branches unaided.

11 Disclosure Classes and Their Bindings to Declared Grounds

This section fixes how access to a record is restricted. Its role is to make restriction a recorded and inspectable act rather than a property of a container, and to bind every restriction to a stated reason that determines what may be withheld. Its structure treats the disclosure class, the four grounds, the binding between them, the per-record condition, and what the protocol declines to specify. Its method is definition followed by requirement, with the analysis of the grounds taken from the companion paper on restriction and not re-argued.

11.1 The Disclosure Class

Definition 11.1. **(Disclosure class).** *A disclosure class is a value attached to a record, governing the set of parties to whom an implementation may disclose it. The protocol requires that classes exist, that they be ordered by inclusion, and that an implementation enforce them. The identity of the classes and the membership of each are declared in an operating charter.*

Two properties are fixed by the protocol and the rest is left to the charter.

Classes are ordered by inclusion, so that a record disclosed at one class is disclosed to every party in the classes that contain it. Without the ordering, scheduled release in the following section has no meaning.

Class values are opaque to the protocol. An implementation enforces them and does not interpret them, which is the operative form of the capacity-and-content requirement. A community with two classes and a community with six are both conformant.

11.2 The Four Grounds

Definition 11.2. **(Ground of restriction).** *A ground is the declared reason for restricting disclosure. Four grounds are fixed by the protocol:* rivalry*, where an instrument or resource admits limited simultaneous use;* hazard*, where propagation of the content creates risk (Bostrom, 2011);* exploratory vulnerability*, where premature exposure would suppress the work; and* appropriability*, where excludability is required to fund the work's production.*

Table 11.1: The four grounds, the object each restricts, and the residue that remains disclosable

Ground	Object restricted	Residue remaining disclosable
Rivalry	Access to the instrument or resource	The trajectory in full
Hazard	The propagable content of a method	Questions, decisions, interpretations, negative results
Exploratory vulnerability	The timing of exposure	Everything, at the scheduled time
Appropriability	Content whose disclosure destroys excludability	Existence, questions, decisions, negative results

The grounds are fixed rather than extensible, for a reason worth stating. Each ground determines a different object of restriction and a different residue that remains disclosable, and a community free to invent grounds could restrict anything by naming a reason.

Table 11.1 states the correspondence. Conditional access of the kind developed for capable computational systems (Shevlane, 2022) is the mechanism the hazard row assumes. The rivalry row is the one implementers most often get wrong: an instrument that admits one user at a time supplies no reason to restrict the record of what was done with it.

11.3 The Binding

Requirement 11.3. (Declared ground). *A record disclosed at less than the widest class available in its trajectory must carry a declared ground. An implementation must reject a restriction that declares no ground, and must present the ground wherever it presents the restriction.*

Requirement 11.4. (Residue). *Where a ground restricts a record, the residue corresponding to that ground must be disclosed at the widest class available. An implementation must not use a ground to withhold material that the ground does not cover.*

The second requirement is what makes the grounds do work. Under hazard, the existence of the work, the questions pursued, the decisions taken and the negative results are disclosed while the propagable method is withheld. Under appropriability, the same residue is disclosed while the content whose disclosure would destroy excludability is withheld. A design without the residue rule would permit any ground to justify total silence, which is the present arrangement with a label attached.

Misapplication has a name and the specification records it. A restriction declaring appropriability where nothing is in fact appropriated is rent-seeking secrecy; a restriction declaring hazard where the content carries no propagable risk is the same act under another name. The protocol cannot detect either, and the requirement that the ground be declared and displayed is what allows a reader to raise the question.

11.4 Restriction Is Per Record

Requirement 11.5. (Per-record disclosure). *A disclosure class attaches to an individual record. An implementation must not make disclosure a property of a repository, a project or a trajectory as a whole, and must permit records within one trajectory to carry different classes.*

This is the second point at which the version-control analogy fails. In a version-control system, openness is a property of a repository chosen by its owner. Research requires that a single line of work carry states disclosed to everyone, states disclosed to a group, and states disclosed to nobody, with the differences declared and grounded.

A consequence for derived views: a view computed for a reader must be computed over the records that reader may see, and an implementation must not disclose the existence of a record by omission, ambiguity or gap in numbering where the charter requires the record's existence to be concealed.

11.5 The Matters Left to the Charter

Three matters are left to the operating charter and are named so that their absence is not read as an oversight.

Which classes exist and who belongs to them. A protocol fixing membership would fix a governance model, and communities differ.

What consent is required before a record concerning a person is created, and what retention applies. These are matters of law and of charter, and the protocol supplies the fields in which the answers are recorded.

And who may change a record's class. The change is an administrative operation and is recorded as one under the requirement of Section 4; who may perform it is a charter question, and the record makes the answer visible after the fact.

11.6 Summary of Results

This section has fixed disclosure as a per-record property bound to a declared reason. Definition 11.1 requires classes to exist, to be ordered by inclusion and to be enforced, while leaving their identity and membership to a charter. Definition 11.2 fixes four grounds and Table 11.1 states, for each, the object restricted and the residue that remains disclosable. Requirement 11.3 requires a ground to be declared for every restriction and Requirement 11.4 requires the residue to be disclosed, which is what prevents a ground from justifying total silence; misapplication is named and is left detectable by readers rather than by the protocol. Requirement 11.5 fixes restriction as per record and forbids repository-level openness, with a consequence for derived views. Membership, consent, retention and the authority to reclassify are left to the charter.

12 Monotone Disclosure and Scheduled Release

This section fixes how a record's disclosure may change. Its role is to discharge the requirement that disclosure be monotone, and to specify the release act by which a state

becomes citable. Its structure treats the monotonicity condition, the default, scheduled release, the release act and what it records, and the emission of an artefact acceptable to the incumbent reward system. Its method is definition followed by requirement, with the consequences for participants stated where they are severe.

12.1 Monotonicity

Requirement 12.1. (Monotone disclosure). *Disclosure may widen and must not narrow. An implementation must not provide an operation that reduces the class of a record already disclosed, and must not represent such a reduction as having occurred.*

The ground is that a reduction cannot be effected. A party who has read a record retains what they read, so an operation appearing to withdraw disclosure conceals from the record's own participants a state of affairs that obtains outside it. An implementation offering the appearance of withdrawal would be trusted with material that should never have been entered.

Two operations resemble narrowing and are distinguished. Redaction removes content and is treated in Section 13; it is a removal from the store and not a change of class. Revocation of a party's membership of a class is a charter matter concerning future disclosures and does not alter what was already disclosed.

12.2 The Default

Requirement 12.2. (Restrictive default). *A record must be created at the most restrictive class applicable under its declared ground, and must widen only by an explicit act. An implementation must not create records at a wider class by default, and must not widen a record's class as a side effect of any other operation.*

The requirement follows from monotonicity. Where disclosure cannot be undone, an error in the direction of openness is uncorrectable, and defaults are where such errors occur.

12.3 Scheduled Release

Definition 12.3. (Scheduled release). *A scheduled release is a recorded commitment that a record will be disclosed at a stated wider class at or after a stated time. The schedule is part of the record and is disclosed at the record's current class.*

Requirement 12.4. (Effect of a schedule). *An implementation honouring a scheduled release must widen the record's class at the stated time without a further act by any party. A schedule may be shortened by an act widening disclosure earlier; it must not be extended or cancelled, and an implementation must record any attempt to do so as an administrative operation with its ground.*

The asymmetry is the substance. Exploratory vulnerability justifies delay and does not justify permanent withholding, and a schedule that could be extended indefinitely is

a permanent withholding that has been made to look temporary. The specification permits the extension attempt to be recorded, because a charter may in some circumstances allow it, and requires that the attempt be visible.

12.4 The Release Act

Definition 12.5. (**Release**). *A release is a transition of the release act declaring that a state is published at a stated class. It records the identifier of the state, the class, the time, and the identifiers of every transition referencing that state whose disposition is unresolved at that moment.*

Requirement 12.6. (**Release records standing objections**). *A release must enumerate the unresolved objections standing against the state at the time of release. An implementation must not omit them, must not permit a release conditional on their resolution, and must not present a release as certifying, validating or approving the state released.*

The requirement is the point at which this specification differs most visibly from publication as presently practised. A release asserts that a state is published and that these objections stand. It asserts nothing about their merit, and a community that treats an enumerated objection as a defect will produce releases with the objections suppressed, which is a failure of the charter and not of the protocol.

A release is also what serves priority. It carries a time and a registrant, and where a community honours registration the release is the object to which a priority claim attaches (Merton, 1957). The protocol supplies the record and cannot supply the recognition.

12.5 Emission of a Citable Artefact

Requirement 12.7. (**Emission on release**). *An implementation must be able to emit, from a release, a document containing the released state, the chain of transitions leading to it, the parties and their contributor roles, the absorption links with their attributions, and the objections standing at release. The emitted document must carry the identifier of the release from which it was generated.*

The requirement discharges the compatibility condition of the companion paper. It allows a participant to obtain, from a conformant record and without additional work, in the manner that staged publication services already provide for outputs (Octopus, 2022), an object of the kind the incumbent reward system accepts, carrying an appendix that no other process can produce. Adoption does not require anyone to abandon publication.

12.6 Summary of Results

This section has fixed how disclosure changes. Requirement 12.1 makes disclosure monotone on the ground that narrowing cannot be effected, and distinguishes redaction and membership revocation from narrowing. Requirement 12.2 fixes the restrictive default

and forbids widening as a side effect. Definition 12.3 and Requirement 12.4 fix scheduled release, which may be shortened and must not be extended or cancelled, with any attempt recorded. Definition 12.5 fixes the release act and Requirement 12.6 requires it to enumerate standing objections, forbids conditioning release on their resolution, and forbids presenting a release as certification. Requirement 12.7 requires an implementation to emit a citable document from a release, carrying the chain, the contributors, the absorptions and the standing objections, which discharges the compatibility requirement.

13 Separability of Content, Redaction, and the Integrity of the Log

This section fixes how a record can be both tamper-evident and lawfully redactable. Its role is to resolve a conflict that has defeated comparable systems: an append-only log with content-derived identifiers cannot admit deletion, and a record holding personal data must admit it (Finck, 2018; European Union, 2016). Its structure treats the separation of the skeleton from the content, redaction, what a redaction leaves, the treatment of derived objects, and the limits of the arrangement. Its method is definition followed by requirement, and it adjudicates no legal question.

13.1 The Skeleton and the Content

Definition 13.1. (**Skeleton and content**). *The* skeleton *of a record consists of its identifier, its kind, its parent links, its state references, its typed fields, its parties, its signature, its disclosure class and ground. The* content *consists of the material the record refers to: the text of a state, a recording, a transcript, an annotation body, a deposited file.*

Requirement 13.2. (**Separability**). *Content must be stored separately from the skeleton and referenced from it by a content-derived identifier. A skeleton must remain valid and verifiable when the content it references has been removed.*

The requirement is what makes the rest of the section possible. The signature covers the skeleton, which includes the identifier of the content; removing the content does not alter the skeleton, so the chain of identifiers and signatures remains intact and every later record remains verifiable.

13.2 Redaction

Definition 13.3. (**Redaction**). *Redaction is the removal of content from the store while the skeleton referencing it remains. A redacted record continues to assert that a transition occurred, by whom, of what type, at what position in the graph, and no longer supplies what was said.*

Requirement 13.4. (**Recording of redaction**). *A redaction must be recorded as an administrative operation carrying the identifier of the record redacted, the time, the party who performed it, and the ground on which it was performed. An implementation must not remove a skeleton, must not remove the record of a redaction, and must not represent redacted content as never having existed.*

The last clause is the substance. A system that erased the trace of an erasure would leave a record that misdescribed its own history, and would make every later reader's inference unreliable in a way they could not detect.

13.3 The Elements Surviving a Redaction

Three things survive a redaction, and an implementation must present all three.

The fact of the transition, with its type, its position in the graph and its date, so that the structure of the trajectory is unimpaired.

The parties, unless the redaction ground requires their removal, in which case the skeleton records that a party field was redacted and the signature over the skeleton is preserved with the party identifier replaced by its own content-derived identifier.

And the fact that a redaction occurred, with its ground, so that a reader who finds a state whose antecedent is unreadable knows why.

Requirement 13.5. (Structural preservation). *A redaction must not alter the graph. Parent links, state references and the ordering they induce must survive redaction unchanged, and an implementation must not remove a record because its content has been removed.*

13.4 Derived Objects and Propagation

Redaction is easy in one store and hard across many, and the specification states the obligation without pretending it is discharged automatically.

Requirement 13.6. (Propagation of redaction). *An implementation that has received records from another implementation must accept and act on a redaction notice referring to those records, and must forward such notices to implementations to which it has propagated them. An implementation must record redaction notices it has received and forwarded.*

Two honest limits. A copy held by a party who has left the arrangement cannot be reached, and no protocol reaches it. And derived objects, including emitted documents and cached views, may embed content that a later redaction removes; an implementation must reconstruct affected caches and must record which emitted documents referenced redacted content, so that a party can pursue them outside the system.

13.5 The Limits of the Arrangement

Three limits are recorded, and the third bears on how the arrangement should be described to participants.

The separation makes redaction possible and does not make it lawful. Whether a given redaction satisfies a given legal obligation is a question the specification does not answer, and the design's contribution is that the question is answerable at all, since a record incapable of redaction forecloses the answer everywhere.

Content-derived identifiers of removed content remain in the skeleton. Where content is short and its space of possibilities small, an identifier may permit reconstruction

by exhaustive search, and an implementation handling such content must apply a per-record secret before computing the identifier.

And a redaction is visible. A participant who removes their material leaves a record that they did so, with a ground. This is the price of a tamper-evident log, it cannot be avoided within one, and participants should be told before they enter rather than after they attempt a removal.

13.6 Summary of Results

This section has fixed the arrangement by which the log is both tamper-evident and redactable. Definition 13.1 separates skeleton from content and Requirement 13.2 requires separate storage with reference by content-derived identifier, so that removal of content leaves the signature chain intact. Definition 13.3 and Requirement 13.4 fix redaction as a recorded administrative operation, forbid removal of skeletons and of redaction records, and forbid representing redacted content as never having existed. Requirement 13.5 preserves the graph across redaction. Requirement 13.6 fixes acceptance and forwarding of redaction notices, with the unreachable copy and the embedded derived object named as limits. Three further limits are recorded: possibility is not lawfulness, short content may be reconstructible from its identifier and requires a per-record secret, and the fact of a redaction is unavoidably visible.

14 Identity, Keys, and Optional Bindings to Institutional Identifiers

This section fixes what a party is, for the purposes of the protocol. Its role is to supply the identity on which attestation, attribution and disclosure all depend, while keeping the requirement low enough that the entry path remains open. Its structure treats the party identifier, the key, optional bindings to external identifiers, assurance levels, and key lifecycle. Its method is definition followed by requirement, with the reasons for refusing a stronger identity requirement stated where the refusal is made.

14.1 The Party

Definition 14.1. (Party). *A party is an entity capable of performing and registering acts, identified within the protocol by a public key. A party identifier is the identifier of that key, and every act, registration, attribution and administrative operation records the party identifier of the entity performing it.*

Nothing in the protocol requires that a party correspond to a natural person, that a natural person hold one party identifier, or that a party identifier be connected to a legal name. What the protocol requires is continuity: the same party identifier across acts is what makes attribution meaningful and what makes attestation by a distinct party checkable.

Claim 14.2. *Attestation and attribution require continuity of identity and do not require legal identity. A key pair establishes that the party who registered this transition is distinct from the party who performed it, and that the same party acted last week, which is the whole*

of what the mechanisms of Section 8 and Section 9 depend on.

14.2 The Refusal of a Universal Identity Requirement

Requirement 14.3. (Pseudonymous participation). *An implementation must permit a party to participate under a key not bound to any external identifier, at every conformance tier. An implementation must not require a legal name, a telephone number, an institutional affiliation or a government identifier as a condition of holding a party identifier.*

Three reasons, of which the second is the one this series cares about.

The mechanisms do not need it, as Claim 14.2 states.

A universal identity requirement excludes the population the companion papers concern. Unaffiliated scholars without institutional credentials, parties in jurisdictions where a telephone number is a state-linked identity document, and parties whose participation carries risk are excluded by such a requirement and by nothing else in the design. The entry path of Section 6 would be closed at the door.

And the empirical record for such requirements is poor, which is an instance of the general point that a technical configuration establishes rules whose costs fall where the configuration was not examined (Reidenberg, 1998; Lessig, 2006). Real-name policies have produced exclusion without the improvement in conduct they were introduced to secure, and a specification adopting one would be adopting a measure whose costs are documented and whose benefits are not.

14.3 Optional Bindings

Communities and individuals will often want more, and the protocol supplies the means without imposing the requirement.

Definition 14.4. (Binding). *A binding is a recorded, verifiable association between a party identifier and an external identifier: a researcher identifier, an institutional account, a domain, or a legal identity attested by a third party. A binding is recorded as an administrative operation and carries the identifier of the party who attested it.*

Requirement 14.5. (Visibility and revocability of bindings). *A binding must be visible wherever the party identifier is presented, must record who attested it and when, and must be revocable by a further recorded operation. An implementation must not present an unbound party identifier as though it were bound, and must not present a revoked binding as current.*

14.4 Assurance Levels

Charters differ in what they demand, and the protocol expresses the difference without deciding it.

Requirement 14.6. (Assurance by class and not by gate). *An operating charter may require a stated assurance level for participation in a given disclosure class or for acts of a given kind. An implementation must enforce such a requirement at the point of the act and must not apply it as a condition of holding a party identifier or of reading records disclosed at the widest class.*

The distinction between a class condition and an entrance gate is the substance. A community handling material restricted on the hazard ground may reasonably require a verified institutional identity for access to that class. The same community must not require it of a stranger who wishes to challenge a claim disclosed publicly, since that is the act by which parties without position enter, and gating it reproduces the bootstrap failure the design exists to avoid.

14.5 Key Lifecycle

Requirement 14.7. (Key rotation and loss). *A party may rotate a key by recording an administrative operation signed by the old key nominating the new one, after which both identifiers refer to the same party and prior records remain valid under the old identifier. Where a key is lost, a party may record a new identifier and a charter-defined procedure may associate the two; an implementation must record which procedure was used and must not represent an association made by such a procedure as equivalent to one signed by the old key.*

The final clause prevents a silent weakening. Recovery procedures are necessary and are weaker than a signature, and a record that presented the two identically would misdescribe the strength of its own evidence.

Compromise is treated as revocation with a date. Registrations made before the recorded compromise remain in the log and remain marked with the compromise, which allows a reader to weigh them; an implementation must not remove them, since removal would alter the graph and would conceal the history the record exists to hold.

14.6 Summary of Results

This section has fixed the party as a key. Definition 14.1 requires continuity and requires nothing about legal identity, and Claim 14.2 states that continuity is all the attestation and attribution mechanisms depend on. Requirement 14.3 permits pseudonymous participation at every tier and forbids requiring legal names, telephone numbers, affiliations or government identifiers, on the grounds that the mechanisms do not need them, that requiring them excludes the population this series concerns, and that the empirical record for such requirements is poor. Definition 14.4 and Requirement 14.5 fix optional bindings as visible, attributed and revocable. Requirement 14.6 permits a charter to demand assurance for a disclosure class or an act kind and forbids applying it as an entrance gate. Requirement 14.7 fixes key rotation by signed nomination, admits charter-defined recovery while requiring it to be marked as weaker, and treats compromise as revocation with a date.

15 Sealed Registration and the Separation of a Claim from Its Disclosure

This section fixes the mechanism by which a party may establish that a state existed at a time without disclosing what it was. Its role is to make the arrangement enterable by a participant who is unwilling to expose work in progress, which the companion paper identifies as the ordinary and rational position of a party without standing. Its structure treats the sealed registration, the opening act, what the mechanism secures, what it does not, and the treatment of independent arrival. Its method is definition followed by requirement, with the limits stated as constraints on what an implementation may claim.

15.1 The Sealed Registration

Definition 15.1. (Sealed registration). *A sealed registration is a record carrying the content-derived identifier of a state, the party identifier of the registering party, a time, and a signature, with the content itself disclosed to no party. The record asserts that the registering party held content with that identifier at that time.*

Requirement 15.2. (Availability at every class). *Registration must be available at every disclosure class, including the class disclosing to no party beyond the performer. An implementation must permit a party to record a trajectory from its first state without disclosing any content, and must not require disclosure as a condition of registration.*

The mechanism requires nothing the specification does not already have. Content-derived identifiers are required by Section 13 for separability, and a sealed registration is the skeleton of a record whose content has not been disclosed rather than a new kind of object.

15.2 The Opening Act

Definition 15.3. (Opening). *An opening is a transition disclosing content previously sealed, recording the identifier of the sealed registration and the content, so that any party may verify that the content yields the identifier registered earlier.*

Requirement 15.4. (Verifiability of an opening). *An implementation must permit any party to whom an opening is disclosed to verify, from the record alone, that the disclosed content yields the identifier in the sealed registration. Where verification fails, the implementation must record the failure and must not remove either record.*

Opening is optional and may never occur. A party may seal a trajectory and abandon it, and the sealed registrations remain as evidence of what was held and when, unopened and unverifiable by anyone.

15.3 The Effects of Sealed Registration

The mechanism separates the making of a claim from the disclosure of its content, which is the separation the incumbent arrangement lacks. Under that arrangement priority

attaches at publication, so the generative phase is unprotected and a party unwilling to disclose has one instrument, which is silence.

Claim 15.5. *Sealed registration permits a record to begin on the first day of a piece of work and to be opened at a time of the party's choosing, with evidence available throughout that the states existed when the record says they did. It converts the choice between disclosing early and recording nothing into a choice about when to disclose.*

15.4 The Limits of Sealed Registration

Five limits are fixed, and the fifth governs how the mechanism should be described.

It evidences possession of a content and not understanding of it. A party may register a state they do not comprehend, and the record distinguishes nothing.

It generates nothing. A sealed state is invisible, so it attracts no objection, no connection and no encounter. The mechanism protects a participant during precisely the phase in which the arrangement's benefits are unavailable to them, and it purchases the possibility of opening later.

It adjudicates nothing against independent arrival. A second party reaching the same state without knowledge of the first has done nothing wrong, and the record shows two registrations with two times and asserts nothing about the relation between them.

Requirement 15.6. (No inference from precedence). *An implementation must not present an earlier sealed registration as establishing that a later party derived their work from it, and must not rank, order or annotate parties by the times of their sealed registrations.*

And a timestamp is not priority. Priority is a community's recognition of a claim, and the recognition is a charter matter and a matter of norms that no specification manufactures. In a community that does not honour registration, the mechanism supplies evidence to a party who has no forum in which the evidence counts.

Requirement 15.7. (Statement of the limit). *An implementation must not describe sealed registration as establishing priority, and must describe it as recording that a party held a content at a time. Where a community honours registration for priority, the honouring is recorded in the operating charter and not in the protocol.*

15.5 Dependence Outside the Protocol

One dependence is recorded because it is easy to overlook. A sealed registration is evidence only if the time it carries is credible to a party who does not trust the registrant. An implementation must therefore anchor sealed registrations in a manner a third party can check, by publishing the identifiers at intervals in a medium the implementation does not control, or by obtaining a timestamp from a party independent of it, following the construction of Haber and Stornetta (1991), and must record which method was used.

15.6 Summary of Results

This section has fixed the separation of a claim from its disclosure. Definition 15.1 fixes the sealed registration as a skeleton whose content is undisclosed, and Requirement 15.2 makes registration available at every class including the class disclosing to nobody. Definition 15.3 and Requirement 15.4 fix the opening act and require verification from the record alone, with failures recorded and nothing removed. Claim 15.5 states what the mechanism secures: a record may begin on the first day and be opened at a chosen time. Five limits are fixed: possession is not understanding, a sealed state generates nothing, independent arrival is not adjudicated, Requirement 15.6 forbids any inference from precedence, and Requirement 15.7 forbids describing the mechanism as establishing priority. Anchoring of the recorded time in a medium the implementation does not control is required and recorded.

16 Propagation of Transitions between Trajectories and Implementations

This section fixes what travels and how. Its role is to discharge the requirement that a record be portable and continuable elsewhere, and to give the word propagation the meaning it carries in the companion paper, which is the movement of transitions and not the dissemination of finished artefacts. Its structure treats the two kinds of propagation, the minimum on which two implementations must agree, continuation under a second implementation, the treatment of records received, and the failure cases. Its method is definition followed by requirement.

16.1 The Two Kinds of Propagation

Definition 16.1. (Propagation). *Propagation is the movement of transitions between trajectories or between implementations.* Lateral *propagation is the entry of a transition raised against one trajectory into another, and the movement of absorbed content across a divergence with its attribution.* Inter-implementation *propagation is the exchange of records between implementations sharing no operator, in the manner of deployed notification and federation protocols* (*Confederation of Open Access Repositories, 2022; Webber et al., 2018*).

Dissemination of finished artefacts is excluded from the term. Repositories, identifier services and indexes perform it, they perform it adequately, and nothing in this specification concerns it.

16.2 The Minimum Agreement

Substrate independence forbids the specification from fixing a wire format, so interoperability must rest on something else, and the specification states what.

Requirement 16.2. (The exchange minimum). *Two implementations exchanging records must agree on the record structure of Section 6, the act and disposition vocabularies, the identifiers of the bound vocabularies in use and their versions, the construction of content-derived identifiers, and the signature scheme. They need agree on nothing else, and in particular need not agree on storage, transport, interface or serialisation.*

The clause on identifier construction and signature scheme is the one that cannot be relaxed. Two implementations computing identifiers differently produce records that cannot be verified across the boundary, and the record's credibility does not survive the crossing.

Requirement 16.3. (Declaration of profile). *An implementation must declare, in a form another implementation can read, the protocol version it implements, the conformance tier, the identifier construction, the signature scheme, and the bound vocabularies with their versions. An implementation receiving records must record the declaration under which they were received.*

16.3 Continuation under a Second Implementation

Portability is not export, and the difference is made operative here.

Requirement 16.4. (Continuation). *A participant must be able to obtain the complete record of a trajectory, comprising its transitions, their skeletons and signatures, their attributions and absorption links, and its parent structure, without the permission of any position holder, and must be able to continue it under an implementation the original operator does not control. Transitions appended after continuation must reference the transitions obtained as parents, so that the continued trajectory is one graph and not two.*

Two consequences follow that an implementer must handle.

Identifiers must continue to resolve, which Requirement **??** already fixes, and a continuation whose references have become unresolvable is an archive rather than a trajectory.

Content disclosed at a restricted class does not travel with the record unless the receiving implementation can honour the class. An implementation transferring a record must either transfer the class and the ground with it and honour them, or transfer only the skeleton and record that content was withheld.

16.4 Records Received from Elsewhere

Requirement 16.5. (Treatment of received records). *An implementation receiving records must verify the signatures it can verify, must retain records whose signatures it cannot verify while marking them unverified, and must not alter received records. Where a received record uses a vocabulary value the implementation does not recognise, the record is retained unchanged and the value is preserved.*

The prohibition on alteration extends to normalisation. An implementation that rewrote received records into its own preferred form would invalidate their signatures and would destroy the property that makes propagation worth having.

Redaction notices are the exception to the rule that received records are not altered, and they are treated in Section 13: a notice is honoured, forwarded, and recorded.

16.5 The Failure Cases

Four are named, and the fourth is the one with no technical answer.

Version mismatch. An implementation receiving records under a protocol version it does not implement must retain them and must not process them as though they were of its own version. Where the versions differ only in additions, it may process the fields it recognises and must record that it did so.

Vocabulary drift. Where two implementations use different versions of a bound vocabulary, each record carries its own version under Requirement 7.3, and no reconciliation is performed by the protocol.

Partial records. An implementation may receive a subgraph whose parents are absent. It must retain the subgraph, must mark the missing parents as unresolved references, and must not synthesise the missing transitions or present the subgraph as complete.

Divergent continuation. Two parties may continue the same obtained record independently, producing two graphs that share a common ancestry and are held by different implementations. This is a divergence under Section 10 and is treated as one: both are retained, neither is principal, and where the parties later exchange records the shared ancestry is visible in the parent links. No reconciliation is required and none is provided.

16.6 Summary of Results

This section has fixed propagation as the movement of transitions and excluded dissemination of artefacts from the term. Definition 16.1 separates lateral from inter-implementation propagation. Requirement 16.2 fixes the exchange minimum, of which identifier construction and signature scheme are the parts that cannot be relaxed, and Requirement 16.3 requires a readable declaration of profile. Requirement 16.4 fixes continuation, requiring appended transitions to reference obtained ones as parents so that a continued trajectory is one graph, with restricted content travelling only where the receiving implementation honours the class. Requirement 16.5 fixes the treatment of received records, forbidding alteration including normalisation. Four failure cases are treated: version mismatch, vocabulary drift, partial records, and divergent continuation, the last of which is a divergence and receives no reconciliation.

17 Durability, Format Stability, and Archival Custody

This section fixes what an implementation must do so that a record survives its tools and its operator. Its role is to discharge the durability requirement, which follows from a claim the companion paper makes about the value of trajectories: an approach abandoned under one set of conditions may become usable when conditions change, and a record whose value appears after a decade must be readable after a decade. Its structure treats the format condition, custody, deposit, succession, and the limits of what a specification can secure. Its method is definition followed by requirement.

17.1 The Format Condition

Requirement 17.1. (Self-describing formats). *The authoritative form of a record must be a plain, self-describing serialisation readable without the software that produced it, in a format whose specification is publicly available. An implementation must not hold the authoritative form of a record only in a database, an index, or a proprietary container.*

An implementation may use any internal representation it likes for speed. What the requirement forbids is that the internal representation be the record, since a record recoverable only through a running system is lost when the system stops running, and systems stop running.

Requirement 17.2. (Field documentation). *An implementation must publish, alongside its records, the identifiers and versions of the vocabularies its records use and the construction of its identifiers and signatures, in a document a reader can consult without access to the implementation.*

The requirement addresses a failure that appears only after the failure is irreparable. A serialisation that is readable but uninterpretable, because the meaning of its fields lived in a codebase, is a durable record of nothing.

17.2 Custody

Definition 17.3. (Custody). *Custody of a record is the responsibility for holding it, keeping it readable, and making it available. Custody is held by a party and not by a system.*

Requirement 17.4. (Distributed custody). *A record must be held by at least two parties who do not share an operator, following the principle of replicated independent custody in digital preservation practice (Maniatis et al., 2005), at the group and open conformance tiers. An implementation must record which parties hold copies and must make the record obtainable by any participant under Requirement 16.4.*

The requirement is modest and is the minimum that survives the loss of one party. It is also what makes portability meaningful in practice, since a right to obtain a record from a party who has ceased to exist is a right without an object.

17.3 Deposit

Requirement 17.5. (Archival deposit). *At the open conformance tier, released states and the transitions leading to them must be deposited with an archival service that operates independently of the implementation and that has stated preservation commitments. The deposit identifier must be recorded in the trajectory.*

The condition applies to released material and not to the whole record, and the restriction is deliberate. Depositing sealed or restricted content with a third party would place material outside the disclosure regime that governs it, which Section 11 forbids in substance.

17.4 Succession

Requirement 17.6. (Succession arrangements). *An implementation operated by an organisation must publish what becomes of the records it holds if the organisation ceases to operate: the party to whom custody passes, or the archival service with which material has been deposited, or the means by which participants may obtain the records. An implementation must not claim durability without publishing such an arrangement.*

The requirement is the point at which this specification touches the custodianship question treated in the companion paper. An organisation that maintains a specification and operates an implementation has an interest in the continuity of both, and the interest is legitimate. What the requirement secures is that the arrangement be stated in advance, so that participants know before they enter whose commitment their record depends on.

17.5 The Limits of Specification

Three limits are recorded.

A specification cannot fund preservation. Deposit services are maintained by parties whose commitments are periodically renegotiated, and a record deposited today depends on decisions that will be made by others.

A specification cannot compel a party to keep a copy. Requirement 17.4 obliges a conformant implementation and reaches nobody who has left the arrangement.

And format stability is a matter of degree. A plain serialisation readable today may require interpretation in decades, and the requirement of published field documentation reduces the difficulty without removing it. What the specification secures is that the difficulty is one of interpretation rather than of recovery.

17.6 Summary of Results

This section has fixed durability. Requirement 17.1 requires the authoritative form of a record to be a plain, self-describing, publicly specified serialisation and forbids holding it only in a database or proprietary container, while permitting any internal representation. Requirement 17.2 requires vocabularies, identifier construction and signature construction to be documented outside the implementation. Definition 17.3 fixes custody as held by a party, and Requirement 17.4 requires at least two custodians without a shared operator at the group and open tiers. Requirement 17.5 requires archival deposit of released material at the open tier, restricted to released material so that deposit does not evade the disclosure regime. Requirement 17.6 requires published succession arrangements and forbids claiming durability without them. Three limits are recorded: a specification cannot fund preservation, cannot compel a departed party, and cannot make interpretation across decades unnecessary.

18 Operating Charters and the Level-1 Interface

This section fixes the interface between the protocol and the normative level. Its role is to specify how a community's rules attach to a record without any of those rules entering

the specification, which is the operative form of the separation of levels. Its structure treats the charter as a document, what it must contain to be usable by an implementation, how records reference it, how it changes, and what the protocol refuses to do with it. Its method is definition followed by requirement, and the specification interprets no charter content.

18.1 The Charter

Definition 18.1. (Operating charter). *An operating charter is a document adopted by a community, in the sense of the collective-choice arrangements by which a group settles its own operational rules (Ostrom, 1990), identified by a persistent identifier and versioned, stating the rules under which that community operates a trajectory or a set of trajectories. The protocol references a charter and interprets nothing in it.*

The charter is where everything contested lives. Which disclosure classes exist and who belongs to them; what consent is required before a record concerning a person is created; what retention applies; what conduct is expected and what follows from a breach; what identity assurance is demanded for which acts; whether registration is honoured for priority; what review templates a community uses; which extended target and trigger values it has declared.

18.2 The Minimum Content of a Usable Charter

The protocol interprets no charter content and does require that certain matters be settled somewhere, since a record referring to a class nobody has defined is uninterpretable.

Requirement 18.2. (Minimum charter content). *A charter referenced by a conformant record must state the disclosure classes in use and their ordering, the parties or roles belonging to each, the assurance level required for each class and for each kind of act, any extended target or trigger values with their definitions, and the procedure by which the charter itself is amended. An implementation must reject a record referencing a charter that does not state these.*

The requirement is a condition of legibility and not a normative demand. It obliges a community to have decided its own questions and settles none of them.

18.3 Reference from Records

Requirement 18.3. (Charter reference). *A trajectory must record the identifier and version of the charter under which it operates. A record whose disclosure class, assurance requirement or extended vocabulary value derives from a charter must record that charter's identifier and version.*

The version is required for the same reason as with bound vocabularies. A record created under one version of a charter was created under the rules that version stated, and a later reader interpreting it under a revised charter would misread it.

18.4 Charter Change

Requirement 18.4. (Effect of charter amendment). *An amendment to a charter applies to records created after the amendment and must not alter the interpretation of records created before it. An implementation must retain the earlier charter versions its records reference, or must record where they may be obtained.*

One case deserves naming because implementations will handle it badly. Where a charter amendment removes a disclosure class, records already carrying that class retain it, and the implementation must continue to enforce the class as the earlier charter defined it. Reclassifying such records would widen disclosure without an act, which Requirement 12.2 forbids, or would narrow it, which Requirement 12.1 forbids.

18.5 The Refusals of the Protocol

Three refusals are recorded, since each names something a reader may expect a specification to supply.

The protocol supplies no model charter, no default charter and no minimum standard of conduct. A specification supplying one would be a specification of governance, and communities that reject the model would be unable to conform to the protocol.

The protocol enforces no charter provision beyond the fields it carries. An implementation enforces disclosure classes because they are protocol fields; it does not enforce conduct rules, and a breach of a charter is a matter for the community that adopted it.

And the protocol adjudicates no conflict between a charter and law. Where a charter provision cannot lawfully be honoured in a jurisdiction, the implementation operating there resolves the conflict and records what it did, and the specification takes no position on the resolution.

18.6 Summary of Results

This section has fixed the interface to the normative level. Definition 18.1 fixes the charter as an identified, versioned document that the protocol references and never interprets, and names what it holds: classes and membership, consent, retention, conduct, assurance, priority recognition, templates and extended vocabulary values. Requirement 18.2 fixes the minimum content a charter must state to be usable, as a condition of legibility rather than a normative demand. Requirement 18.3 requires records to reference charter identifier and version. Requirement 18.4 makes amendments prospective, requires earlier versions to be retained or locatable, and treats the removal of a class as leaving existing records under the earlier definition. Three refusals are recorded: no model charter, no enforcement beyond protocol fields, and no adjudication between charter and law.

19 Amendment Procedures and Version Negotiation

This section fixes how the specification itself changes. Its role is to make the protocol revisable by a procedure that is recorded and inspectable, since a specification without one either ossifies or is revised by whoever holds informal power. Its structure treats

the versioning scheme, the amendment procedure, what an implementation declares, negotiation between versions, and the limits of what the procedure secures. Its method is definition followed by requirement, and the section applies its own subject matter to itself.

19.1 Versioning

Definition 19.1. (**Protocol version**). *A protocol version identifies a state of this specification. A version is* compatible *with an earlier one where every record valid under the earlier version is valid under it and carries the same meaning; otherwise it is* incompatible.

Requirement 19.2. (**Version identification**). *Every record must carry the identifier of the protocol version under which it was created. An implementation must not alter the version identifier of a record it did not create, and must not upgrade records to a later version.*

The prohibition on upgrading is the substance. A record created under one version asserts what that version's fields meant, and rewriting it under a later version would silently change what it asserts, which is the failure the append-only condition exists to prevent.

Protocol versions are distinct from implementation versions and from charter versions, and an implementation must not use one numbering for two of them.

19.2 The Amendment Procedure

Requirement 19.3. (**Recorded amendment**). *An amendment to this specification is proposed, discussed and adopted through a record conforming to the specification. A proposal is a state; objections to it are challenges; revisions are transformations; adoption is a release at the widest class. The trajectory of the specification must be publicly readable and must remain so.*

The requirement is the reflexive demonstration the companion paper anticipates. It has a practical effect beyond the demonstration: a reader can see which objections were raised against a provision, which were answered, and which stood unresolved at adoption, and Requirement 12.6 obliges the release to enumerate the last of these.

Requirement 19.4. (**Custodial separation**). *The party maintaining this specification must not be the operator of any implementation for which conformance confers advantage, or must record the conflict and the arrangements limiting it. The specification must be licensed so that any party may fork it, and conformance to a fork must be expressible in the version identifier.*

The requirement follows the argument of the companion paper about positions and their bounds. Authority over a specification is bounded by the cost of leaving it (Hirschman, 1970), and a specification that could not be forked would be a position rather than a service.

19.3 Declaration and Negotiation

Requirement 19.5. (Declaration). *An implementation must declare the protocol versions it can read and the version under which it creates records. Where two implementations exchange records, each must record the versions declared by the other at the time of exchange.*

Negotiation is minimal by design. An implementation receiving records under a version it cannot read retains them and marks them unread, as Section 16 requires. There is no translation, no negotiation of a common subset, and no downgrade, because each of these would produce records asserting something other than what their creators asserted.

19.4 The Limits of the Procedure

Three limits are recorded, and the second is the one no procedure removes.

An amendment procedure does not make an amendment good. It makes the objections visible and the adoption dated, and the quality of the result depends on who participates.

The procedure is itself capturable. A party controlling participation in the specification's trajectory controls the specification, and no clause within the specification prevents this; the tendency is documented for voluntary associations whose rules are explicitly egalitarian (Michels, 1962; Shaw and Hill, 2014). What bounds it is the licence to fork and the portability of the specification's own record, which is the same bound the protocol places on every other position and is equally partial.

And an amendment cannot repair a record. Where an amendment corrects a defect, records created under the defective version remain as they were, and the specification must state, for each incompatible change, what a reader should understand about records created earlier.

19.5 Summary of Results

This section has fixed how the specification changes. Definition 19.1 separates compatible from incompatible versions, and Requirement 19.2 requires every record to carry its protocol version and forbids upgrading records, which would silently alter what they assert. Requirement 19.3 conducts amendment through a conforming record, so that objections and their dispositions at adoption are visible, which is the reflexive demonstration the companion paper anticipates. Requirement 19.4 separates custodianship from operation, or requires the conflict to be recorded, and requires the specification to be forkable with forks expressible in the version identifier. Requirement 19.5 fixes declaration, with no translation, negotiation or downgrade permitted. Three limits are recorded: procedure does not confer quality, the procedure is capturable and is bounded only by forkability, and an amendment cannot repair records created earlier.

Table 20.1: The conformance tiers and the requirements each adds

Tier	Adds	Available to its adopter
Personal	Transitions with parent links and identified prior states; append-only log with derived views; three acts as a minimum, namely claim, challenge and release; content separable from skeleton	A searchable record of what was ruled out and why; a list of objections not yet answered; a citable release with its chain
Group	Attestation by a distinct party; contributor roles; absorption links; disclosure classes with declared grounds; administrative operations recorded	Evidence, since credibility begins where a second party registers; visible attribution of changes; onboarding of a new participant from the record
Open	Propagation with declaration of profile; distributed custody; archival deposit of released material; charter reference; identifier resolution independent of the operator	Exchange with parties elsewhere; continuation of the record beyond the operator; entry of strangers through the open register

20 Conformance Tiers and the Minimal Implementation

This section fixes what an implementation must satisfy to claim conformance. Its role is to make the specification adoptable at a scale a single participant can reach, since a protocol that pays only at large numbers is never begun. Its structure treats the three tiers, what each requires, the test for the exclusions, the manner of declaration, and what a conformance claim does and does not license. Its method is definition followed by requirement, with the tiers stated so that each is worth adopting at its own scale.

20.1 The Three Tiers

Definition 20.1. (**Conformance tiers**). *The* personal *tier supports one participant. The* group *tier supports a set of participants sharing an arrangement. The* open *tier supports exchange with implementations operated by other parties.*

The tiers are cumulative: an implementation conforming at a tier satisfies every requirement of the tiers below it. Table 20.1 states the content.

20.2 The Personal Tier and Its Sufficiency

Requirement 20.2. (**Minimal implementation**). *A personal-tier implementation must support the acts claim, challenge and release, parent links, identified prior states, the append-only log with derived views, and separability of content. It must not require attestation, contributor roles, disclosure classes, charters or propagation.*

The tier exists because of a condition the companion paper establishes: an arrangement whose benefits appear only at scale is never adopted, since no participant is ever in a position to obtain them. What a single participant obtains here is a record of what was tried and abandoned with the reasons, a standing list of unanswered objections, and a release that emits a citable document with its chain.

Requirement 20.3. (Marking of the personal tier). *A personal-tier implementation must mark every transition unattested and must state, wherever it presents or exports a record, that the record carries no attestation. It must not describe a personal-tier record as verified, corroborated, or independently recorded.*

The requirement states the honest position: the personal tier delivers utility and no evidential weight, and the two must not be confused at the point where most participants will meet the protocol first.

20.3 The Test for the Exclusions

Three exclusions are the ones an implementation is most likely to violate while acting in good faith, and each is given a test an assessor can apply.

Requirement 20.4. (Test for the scalar exclusion). *An implementation fails conformance where it computes, stores, displays or exports any total order or numeric measure over participants or over trajectories, including counts of transitions, counts of attestations, contribution shares, activity indices and derived scores. Counts of records within a single trajectory, presented without comparison across participants or trajectories, do not violate this requirement.*

Requirement 20.5. (Test for independence). *An implementation fails conformance where any conformant operation cannot be performed without an analytical capability. The test is whether the implementation's record can be created, registered, read, verified, exported and continued using a text editor and a version-control system.*

Requirement 20.6. (Test for the byproduct condition). *An implementation fails conformance where it requires an act whose only purpose is that a record should exist. The test is whether each required act has a purpose for the party performing it that is stated in the implementation's documentation.*

20.4 Declaration

Requirement 20.7. (Self-declaration). *An implementation claiming conformance must publish a declaration stating the protocol version, the tier, the identifier and signature constructions, the bound vocabularies with versions, the substrates supplying storage, identifiers and transport, and the succession arrangement required at the open tier. The declaration must be readable without access to the implementation.*

Conformance is self-declared and is checkable from the record. No certification body is established here, and none is required: a party doubting a declaration may obtain a record under the continuation requirement and verify the signatures, the identifier construction and the presence of the required fields.

20.5 The Scope of a Conformance Claim

Three limits are recorded as constraints on what an implementation may say of itself.

A conformance claim concerns the implementation and not the records it holds. A conformant implementation may hold a fabricated trajectory, and the mechanisms of this specification raise the cost of fabrication without eliminating it.

A conformance claim concerns structure and not quality. Nothing in this specification assesses content, and an implementation must not present conformance as bearing on the value of what its records contain.

And a conformance claim at the open tier does not establish that exchange has occurred. It establishes that the implementation is capable of it, and a reader wanting to know whether a record has travelled should look for the declarations recorded under Requirement 16.3.

20.6 Summary of Results

This section has fixed three cumulative tiers. Definition 20.1 names them and Table 20.1 states what each adds and what its adopter obtains. Requirement 20.2 fixes the minimal implementation at three acts, parent links, identified prior states, an append-only log with derived views, and separability, and requires none of attestation, roles, classes, charters or propagation. Requirement 20.3 requires personal-tier records to be marked unattested wherever presented or exported. Three tests are fixed for the exclusions most likely to be violated in good faith: the scalar test, which permits within-trajectory counts and forbids comparison across participants or trajectories; the independence test, stated as whether the record can be created, verified and continued with a text editor and a version-control system; and the byproduct test, stated as whether each required act has a purpose for the party performing it. Requirement 20.7 fixes self-declaration, with no certification body and verification available from the record. Three limits are recorded: conformance concerns the implementation and not its records, concerns structure and not quality, and at the open tier establishes capability and not use.

21 A Deployment over Version Control and Existing Repositories

This section describes one way of satisfying the specification with systems that exist. Its role is to show that conformance requires no new software, and to expose the points at which a plausible deployment strains against the requirements. Its structure treats the status of the description, the mapping of the record onto a version-controlled repository, the handling of the event plane, the handling of disclosure, propagation and deposit, and the four points at which this deployment is weakest. Its method is description, and nothing in the section is normative: an implementation departing from every choice here may conform, and an implementation following every choice may fail to.

21.1 The Status of This Description

Requirement 21.1. (Non-normative status). *Nothing in this section adds to or qualifies the requirements of this specification. Conformance is assessed against the requirements alone, and a deployment described here conforms only insofar as it satisfies them.*

The section is included because a specification with no worked deployment invites the reply that it cannot be implemented, and excluded from the normative text because a deployment ages faster than a specification and should not date it.

21.2 The Record in a Version-Controlled Repository

A trajectory is a repository. Each transition is a commit, and its skeleton is a plain text file added by that commit, holding the fields of Section 6: the act, target, relation, trigger and disposition; the identifiers of the prior and posterior states; the performing and registering parties; absorption links; the disclosure class and its declared ground; and references to artefacts.

Content is stored in separate files, referenced from the skeleton by the version-control system's own content identifiers. The separation required for redaction is therefore the substrate's ordinary behaviour: removing a content file leaves the skeleton and its references intact and verifiable.

Parent links are the commit's parents, and the acyclicity condition is the substrate's. A synthesis is a commit with several parents and is not a merge in the substrate's sense, since no automatic combination is performed; the performing party composes the posterior state and the commit records it.

Signatures are the substrate's commit signatures, which already cover the commit content and its parents. Attestation is expressed by a commit whose signer differs from the party recorded in the skeleton's performer field, and an implementation enforces the distinction by refusing commits where the two coincide at the group tier and above.

21.3 The Event Plane

Events are held outside the trajectory repository, in whatever systems the participants already use: a calendar, a meeting recorder, a message archive, the working repository of a piece of software. The implementation reads them, proposes candidate transitions, and writes nothing to the trajectory until a party registers one.

The privacy requirement is satisfied by the arrangement rather than by a mechanism: the event plane is never copied into the trajectory, so publishing a trajectory discloses no event record. What is disclosed is a trigger reference in a registered transition, which names the occasion and is disclosed at that transition's class.

21.4 Disclosure, Propagation and Deposit

Disclosure classes are the weakest part of this deployment and are treated in the following subsection. The workable arrangement is one repository per class, with records at more restrictive classes held in repositories with narrower access and referenced from

the wider ones by identifier alone, so that a reader at a wide class sees that a record exists and does not see its content.

Propagation is the substrate's clone and fetch. A participant obtains a complete record by cloning, and continues it by committing to their own copy, with the obtained commits as parents. Exchange between implementations is the same operation, and the declaration of profile is a file in the repository.

Deposit of released material is a transfer of an archive of the repository at a release to a repository service that issues a persistent identifier, with the identifier recorded in a subsequent administrative operation.

21.5 The Weaknesses of This Deployment

Four points, stated because a reader will otherwise assume the mapping is clean.

Disclosure is per repository and the specification requires it per record. The one-repository-per-class arrangement approximates the requirement and does not meet it, since a record whose class changes under a scheduled release must be moved rather than relabelled, and the movement is visible in a way a relabelling would not be. An implementation taking this route must record the movement as an administrative operation.

Identifier resolution depends on hosting. The substrate's content identifiers are stable and are not resolvable by a party who does not hold a copy, so an implementation must obtain resolvable identifiers from a repository service for every state that is referenced from outside, and must record the correspondence.

Redaction is available and is not simple. Removing a content file from the working tree leaves it in the history, so a redaction requires the substrate's history-rewriting facilities or a design in which content files are held outside the versioned tree and referenced by identifier. The second is preferable and is the arrangement assumed above.

Signature coverage is fixed by the substrate. Commit signatures cover the commit's content, which includes the skeleton file with its disclosure class, so the class cannot be widened by a scheduled release without invalidating the signature. The workable arrangement records the class in a separate file that the signature does not cover, which satisfies Requirement 8.3 and requires the implementer to notice the problem.

21.6 Summary of Results

This section has described one deployment and has fixed its status as non-normative in Requirement 21.1. A trajectory is a repository, a transition is a commit carrying a plain-text skeleton, content is held in separate files referenced by identifier, parent links and acyclicity are the substrate's, and attestation is expressed by a signer distinct from the recorded performer. The event plane is held outside the trajectory and is never copied into it, which satisfies the privacy requirement by arrangement. Disclosure is approximated by one repository per class, propagation is clone and fetch, and deposit is an archive transfer recorded by an administrative operation. Four weaknesses are stated: disclosure is per repository rather than per record and class changes require recorded movement; content identifiers are not resolvable without a copy, so resolvable identifiers

must be obtained separately; redaction requires content to be held outside the versioned tree; and commit signatures cover the disclosure class unless it is held in a separate uncovered file.

22 Worked Examples in Mathematics, Machine Learning, and Philosophy

This section works three trajectories through the specification. Its role is to show which requirements are exercised by ordinary research activity, and to expose the places where a participant would find the protocol demanding. Its structure is three cases followed by what they establish jointly. Its method is illustration: the cases are constructed rather than observed, and they establish nothing empirical.

22.1 A Trajectory in Mathematics

Case 22.1. (**An obstruction and an entering stranger**). *A researcher records a question and a claim: that a theorem holds under a stated condition. A first attempt is recorded as a transformation with trigger* reflection*, and a verification records that the attempt fails because an independence assumption does not hold. The researcher records a decision abandoning the approach, with the reason. A later state records the current obstruction, and its disposition is* unresolved*.*

Some months afterwards a party with no prior act in the trajectory finds the unresolved state through the open register, and records a connection to a construction in another field where the same obstruction was met. The trajectory's participants accept the connection; a transformation follows, registered by a party other than its performer, and the entering party is recorded as the performer of the connection with a contributor role.

The case exercises the register of unresolved states, the entry path, attestation, and the decision act. It also shows what the decision act is for: the abandoned approach is interpretable years later only because the reason was recorded with it, and the connection was possible only because the obstruction was addressable as a state.

At the personal tier the same trajectory is recordable and carries no attestation, and the entering party has nothing to find.

22.2 A Trajectory in Machine Learning

Case 22.2. (**An evaluation failure and a restricted method**). *A group records a hypothesis about a training procedure and a method state describing it. Experiments are recorded as verifications with trigger* experiment*, and one records an outcome the hypothesis does not predict. A challenge is registered against the hypothesis by a member who did not propose it; the disposition is accepted, and a transformation records the revised hypothesis with relation* modification*.*

Part of the method is held at a restricted class on the appropriability ground, since the group's funding depends on excludability. The residue is disclosed at the widest class: the existence of the work, the question, the decisions taken, the evaluation outcomes that failed, and the revised hypothesis. A scheduled release is recorded for the restricted state at a stated

date.

A second group, working on a different problem, absorbs the abandoned first hypothesis with its recorded failure, and the absorption link credits the originating party.

The case exercises disclosure classes with a declared ground, the residue rule, scheduled release, and absorption across trajectories. It also shows the residue rule doing the work it exists for: the group withholds what excludability requires and discloses everything the ground does not cover, which is what makes the abandoned hypothesis available to the second group at all.

22.3 A Trajectory in Philosophy

Case 22.3. (A divergence that is not resolved). *A state holds that trust is a property obtaining between individuals. A challenge is registered by a second party, holding that the account omits institutional power. The proposing party accepts and a transformation produces a state in which trust is a process shaped by asymmetry of power.*

A third party challenges the revised state on different grounds, holding that the revision has made the account unable to distinguish trust from compliance. The challenge is neither accepted nor rejected; its disposition is recorded as unresolved*, and work proceeds.*

Two transformations then diverge from the revised state: one narrows the account to institutional settings, the other retains the general scope and weakens the claim. Neither branch is designated principal. A later synthesis draws on both, referencing them as parents, and the branches continue.

A release is declared on the narrowed branch, enumerating the unresolved challenge as standing at the moment of release, and an emitted document carries the state, its chain, the contributors and the standing objection.

The case exercises the unresolved disposition, divergence without merge, synthesis with several parents, and release with enumerated objections. It is the case in which the specification differs most sharply from publication as practised, since the released document declares an objection its author has not answered.

22.4 The Findings Common to the Three Cases

Three observations, and the third is a cost.

Every act in the three cases is one a participant performs for a reason of their own. Nobody documents in order to document: a challenge is raised because the challenger wants the claim changed, a decision is recorded because the group needs to remember what it ruled out, a release is declared because the author wants something citable.

The requirements exercised differ by field, and the tiers accommodate the difference. The philosophical case needs no disclosure classes; the machine-learning case needs them and needs the residue rule; the mathematical case needs the open register and the entry path.

And the demanding part is the same in all three. Recording a decision with its reason, at the moment of abandoning a direction, is work a researcher has no present incentive

to perform, and it is the act on which the reuse of abandoned material depends. The specification makes the act cheap and cannot make it attractive, which is a limit of a protocol and is the subject of the companion paper on incentives.

22.5 Summary of Results

This section has worked three trajectories. The mathematical case exercises the register of unresolved states, the entry path, attestation and the decision act, and shows that an abandoned approach is interpretable later only where its reason was recorded. The machine-learning case exercises disclosure classes with a declared ground, the residue rule, scheduled release and absorption across trajectories, and shows the residue rule making an abandoned hypothesis available while excludability is preserved. The philosophical case exercises the unresolved disposition, divergence without merge, synthesis and release with enumerated standing objections. Jointly the cases show that every act has a purpose for its performer, that the tiers accommodate different fields, and that the decision act is the demanding one, being work with no present incentive on which the reuse of abandoned material nevertheless depends.

23 Failure Modes, Attacks, and Their Treatment

This section states how the specification fails and what it does about each failure. Its role is to prevent the specification from being read as offering guarantees it does not offer, and to give implementers the list of conditions they must handle. Its structure treats failures of the record, failures of position, failures of disclosure, failures of the arrangement, and the failures that have no technical answer. Its method is enumeration: each item states what the failure is, what the protocol does, and what it does not.

23.1 Failures of the Record

Collusive fabrication. Parties who coordinate produce a mutually attested trajectory describing work that did not occur. The protocol raises the cost in proportion to the number of independent registering parties and to the visibility of their other work, and it does not eliminate the failure. An implementation must not describe an attested record as proof of authenticity.

Retroactive alteration. A party alters an earlier transition. Content-derived identifiers over payload and parents make the alteration invalidate every descendant, so the failure is detectable by any holder of a copy. Where a party holds the only copy, detection requires a second custodian, which Requirement 17.4 supplies at the group tier and above.

Backdating. A party records a time earlier than the act. Signatures do not establish time, and the anchoring required in Section 15 moves the question to a party the registrant does not control. Without anchoring, recorded times are assertions.

Flooding. A party registers many low-content transitions. The protocol offers no rate limit and no quality condition, and the mechanisms that bear on the failure are indirect: attestation requires recruiting a registrar, and the exclusion of scalar measures removes the reward for volume. Rationing of attention, whose burden falls on a small number of

maintaining parties (Eghbal, 2016), is a charter matter and is treated in the companion paper on collective action.

23.2 Failures of Position

Registrar gatekeeping. A steward declines to register transitions from a particular party. The protocol records the declining party's other registrations and supplies no remedy. What responds is the charter, and what bounds the position is the portability of the record: the excluded party may obtain it and continue elsewhere.

Custodial capture. An operator restricts access to the record while claiming conformance. The continuation requirement is the test, and an operator refusing it is nonconformant. The protocol cannot compel compliance, and a party's recourse is the copies held under distributed custody.

Capture of the specification. A party controlling participation in the specification's own trajectory controls the specification. No clause prevents this, and the bounds are the licence to fork and the portability of the specification's record, which are partial.

23.3 Failures of Disclosure

Misapplied grounds. A restriction declares appropriability where nothing is appropriated, or hazard where no propagable risk exists. The protocol requires the ground to be declared and displayed and cannot assess it. The failure is detectable by readers and is corrected, where it is corrected, by the community.

Disclosure by inference. A record's existence is revealed by a gap, an identifier sequence or the shape of a derived view. An implementation must compute views over the records a reader may see and must not disclose existence by omission where a charter requires existence to be concealed. This is a genuine implementation hazard and is easy to introduce accidentally.

Redaction that cannot reach. A copy held by a party who has left the arrangement is unreachable by any notice. The protocol requires notices to be accepted and forwarded and reaches nobody outside.

Reconstruction from an identifier. Where redacted content was short and drawn from a small space, its content-derived identifier permits recovery by search. The per-record secret required in Section 13 answers this and must be applied before the identifier is computed, which an implementer must decide in advance.

23.4 Failures of the Arrangement

Surveillance through the event plane. The plane that makes capture cheap is a monitoring log. The privacy requirement confines it to participants and forbids export and indexing, and an implementation that violated it would produce exactly the instrument the design exists to avoid. This is the failure with the largest consequence for participants and the one an implementer is most likely to introduce while improving a feature.

Scores at the edge. An application computes a contribution index over conformant records. The protocol forbids the implementation from doing so and cannot forbid third

parties, and a widely adopted external score would reintroduce the dynamic the exclusion exists to prevent (Espeland and Sauder, 2007). The protocol's only contribution is that it supplies no such quantity itself and no privileged basis for one.

Reflexive confirmation. Registration becomes a habitual click and ceases to carry judgement. The requirement of an affirmative act is the protocol's answer and is weak; what strengthens it is a charter under which registering carelessly is a recognisable failure, and the record shows who registered what.

23.5 The Failures with No Technical Answer

Three, stated so that no reader mistakes the specification's silence for a solution.

A community that punishes disclosed failure will produce records with the failures removed, and the reuse of abandoned material, which is one of the two concrete benefits claimed for the design, will not occur.

A community that does not honour registration will leave the sealed-registration mechanism supplying evidence to parties who have no forum in which it counts.

And a community in which nobody registers another party's transitions will operate at the personal tier under a group-tier declaration, producing records whose marking is correct and whose evidential value is absent. The protocol makes the condition visible in the record and does nothing further.

23.6 Summary of Results

This section has enumerated the failures. Of the record: collusive fabrication, made costly and not impossible; retroactive alteration, detectable through content-derived identifiers given a second custodian; backdating, answered only by anchoring; and flooding, addressed indirectly through attestation and the absence of any reward for volume. Of position: registrar gatekeeping, custodial capture and capture of the specification, all bounded by portability and forkability and none prevented. Of disclosure: misapplied grounds detectable by readers, disclosure by inference as an implementation hazard, redaction that cannot reach departed copies, and reconstruction from an identifier answered by a per-record secret. Of the arrangement: surveillance through the event plane, which is the failure with the largest consequence for participants; external scores, which the protocol can only decline to supply; and reflexive confirmation, which the affirmative-act requirement answers weakly. Three failures have no technical answer: punishment of disclosed failure, non-recognition of registration, and an arrangement in which nobody attests.

24 The Boundaries of the Specification and Its Open Issues

This section states what the specification does not do and what remains unsettled. Its role is to close the paper with its limits rather than its claims, and to name in advance what would show the specification inadequate. Its structure treats what the protocol declines to do, the open engineering questions, the open normative questions, the conditions that would falsify the design, and the status of this version. Its method is enumeration, and

the section makes no claim.

24.1 The Refusals of the Specification

Five refusals, each of which a reader may have expected the specification to reverse.

It does not assess content. No requirement bears on whether a claim is true, whether a challenge is sound, or whether a synthesis is an improvement, and every mechanism that might be mistaken for assessment is a record of what a party did.

It does not measure capacity. A trajectory records what occurred and not what the relation producing it could have produced, and what it records is in any case the articulable fraction alone (Collins, 2010), and the companion paper establishes that two trajectories may coincide in their recorded appearance while differing entirely in this respect.

It does not resolve disputes. Contested attribution, withdrawn attestation and disagreement over a disclosure ground are all recorded, and no party is empowered by the specification to decide any of them.

It does not guarantee lawfulness. The separability and redaction requirements make lawful operation possible where a record incapable of redaction would foreclose it, and whether a given implementation satisfies a given obligation is a question for the implementer and for counsel.

It does not produce encounters. It records them, and it makes states addressable so that a party elsewhere may find one. Whether such finding occurs is a matter of adoption, of scale and of the behaviour of communities, on which the evidence available is discouraging and is stated in the companion paper.

24.2 Open Engineering Questions

Four, each with the condition under which it would be settled.

Per-record disclosure over substrates whose access control is per repository. The deployment of Section 21 approximates the requirement and does not meet it, and a substrate supporting per-object access with cryptographic enforcement would settle the question.

Resolvable identifiers at low cost. Every state referenced from outside its trajectory requires an identifier that resolves independently of its operator, and issuing such identifiers at the granularity of a state is presently expensive. An identifier scheme suited to fine-grained, high-volume objects would settle it.

Redaction across implementations that no longer communicate. Notices reach only implementations that receive them, and no mechanism reaches a copy held by a departed party.

Representation of a state for structural matching across vocabularies. This is the capability on which the cross-field claim of the companion paper depends, it is a research problem of the kind literature-based discovery has pursued for published claims (Swanson, 1986), and the specification supplies a substrate for it while claiming nothing about it.

24.3 Open Normative Questions

Three, none of which a specification can settle.

Whether registration is honoured for priority, which is a matter of what communities recognise.

Whether disclosed failure can be non-punitive, on which the reuse argument depends entirely.

Whether the labour of registering another party's transitions is recognised, since it is unpaid work performed for another's benefit and is the disparity that defeated every comparable system.

24.4 The Findings That Would Show the Specification Inadequate

Four findings, stated so that a reader may look for them.

Adoption at the personal tier without progression to the group tier, over a substantial population and period, would show that the arrangement supplies private utility and does not produce the attested records on which every evidential claim rests.

Records in which the decision act is rare, while transformations are common, would show that the act on which reuse depends is not being performed, and that making it cheap was insufficient.

Widespread declaration of the group tier alongside registration patterns in which performers and registrars are drawn from pairs who register only each other would show attestation operating as a formality.

And the emergence of a widely used external score computed over conformant records would show that the exclusion of scalar measures displaces the dynamic rather than preventing it.

24.5 The Status of This Version

This specification is version 0.1. It has no implementation, no deployment history and no evidence of use, and every requirement in it is a proposal about what such use would need.

The amendment procedure of Section 19 exists because the author expects the requirements to be wrong in ways argument cannot anticipate, and the procedure is conducted through a record conforming to the specification so that the corrections and the objections that produced them are visible to anyone who later asks why a provision reads as it does.

24.6 Summary of Results

This section has fixed the boundaries. The protocol declines to assess content, to measure capacity, to resolve disputes, to guarantee lawfulness, and to produce encounters. Four engineering questions remain open: per-record disclosure over per-repository substrates, resolvable identifiers at state granularity, redaction across implementations that no longer communicate, and the representation of a state for structural matching. Three

normative questions remain open and lie outside what a specification can settle: recognition of registration for priority, the treatment of disclosed failure, and recognition of the labour of registering. Four findings would show the specification inadequate: personal-tier adoption without progression, rarity of the decision act, reciprocal registration pairs, and the emergence of a widely used external score. The version is 0.1, no implementation exists, and the amendment procedure is the means by which the requirements are expected to be corrected.

References

Liz Allen, Alison O'Connell, and Veronique Kiermer. How can we ensure visibility and diversity in research contributions? how the contributor role taxonomy (CRediT) is helping the shift from authorship to contributorship. *Learned Publishing*, 32(1):71–74, 2019.

Nick Bostrom. Information hazards: A typology of potential harms from knowledge. *Review of Contemporary Philosophy*, 10:44–79, 2011.

Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, Internet Engineering Task Force, 1997.

Simon Buckingham Shum, Enrico Motta, and John Domingue. ScholOnto: An ontology-based digital library server for research documents and discourse. *International Journal on Digital Libraries*, 3(3):237–248, 2000.

Harry Collins. *Tacit and Explicit Knowledge*. University of Chicago Press, 2010.

Confederation of Open Access Repositories. The COAR notify protocol. Technical specification, 2022.

Jeff Conklin and Michael L. Begeman. gIBIS: A hypertext tool for exploratory policy discussion. *ACM Transactions on Information Systems*, 6(4):303–331, 1988.

Nadia Eghbal. Roads and bridges: The unseen labor behind our digital infrastructure. Technical report, Ford Foundation, 2016.

Wendy Nelson Espeland and Michael Sauder. Rankings and reactivity: How public measures recreate social worlds. *American Journal of Sociology*, 113(1):1–40, 2007.

European Union. Regulation (EU) 2016/679 of the european parliament and of the council (general data protection regulation), 2016.

Michèle Finck. Blockchains and data protection in the european union. *European Data Protection Law Review*, 4(1):17–35, 2018.

Jonathan Grudin. Why CSCW applications fail: Problems in the design and evaluation of organizational interfaces. In *Proceedings of the 1988 ACM Conference on Computer-Supported Cooperative Work*, pages 85–93, 1988.

Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.

Michael A. Heller and Rebecca S. Eisenberg. Can patents deter innovation? the anticommons in biomedical research. *Science*, 280(5364):698–701, 1998.

Albert O. Hirschman. *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States*. Harvard University Press, 1970.

International Committee of Medical Journal Editors. Recommendations for the conduct, reporting, editing, and publication of scholarly work in medical journals, 2023.

Werner Kunz and Horst W. J. Rittel. Issues as elements of information systems. Technical Report Working Paper 131, Institute of Urban and Regional Development, University of California, Berkeley, 1970.

Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006.

Petros Maniatis, Mema Roussopoulos, T. J. Giuli, David S. H. Rosenthal, and Mary Baker. The LOCKSS peer-to-peer digital preservation system. *ACM Transactions on Computer Systems*, 23(1):2–50, 2005.

Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378. Springer, 1988.

Robert K. Merton. Priorities in scientific discovery: A chapter in the sociology of science. *American Sociological Review*, 22(6):635–659, 1957.

Robert Michels. *Political Parties: A Sociological Study of the Oligarchical Tendencies of Modern Democracy*. Free Press, 1962. First published 1911.

Luc Moreau and Paul Groth. *Provenance: An Introduction to PROV*. Morgan and Claypool, 2013.

Octopus. Octopus: Built for researchers. `https://www.octopus.ac`, 2022. Supported by UK Research and Innovation and the UK Reproducibility Network.

Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.

Joel R. Reidenberg. Lex informatica: The formulation of information policy rules through technology. *Texas Law Review*, 76(3):553–593, 1998.

Robert Sanderson, Paolo Ciccarese, and Benjamin Young. Web annotation data model. W3C Recommendation, 2017.

Aaron Shaw and Benjamin Mako Hill. Laboratories of oligarchy? how the iron law extends to peer production. *Journal of Communication*, 64(2):215–238, 2014.

Toby Shevlane. Structured access: An emerging paradigm for safe AI deployment. *arXiv preprint arXiv:2201.05159*, 2022.

David Shotton. CiTO, the citation typing ontology. *Journal of Biomedical Semantics*, 1 (Suppl 1):S6, 2010.

Stian Soiland-Reyes et al. Packaging research artefacts with RO-Crate. *Data Science*, 5(2): 97–138, 2022.

Michael Spence. Job market signaling. *The Quarterly Journal of Economics*, 87(3):355–374, 1973.

Don R. Swanson. Fish oil, Raynaud's syndrome, and undiscovered public knowledge. *Perspectives in Biology and Medicine*, 30(1):7–18, 1986.

Christopher Lemmer Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. ActivityPub. W3C Recommendation, 2018.

Mark D. Wilkinson et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.

World Intellectual Property Organization. Berne convention for the protection of literary and artistic works, 1979. As amended; Article 6bis on moral rights.